

操作系统，2026

第12讲 设备管理(2)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

典型的输入/输出设备

02

Windows的设备管理

03

Linux的设备管理

典型的输入/输出设备

- 时钟
- 人机交互设备——键盘、鼠标、显示器
- 存储设备——硬盘、SSD

时钟

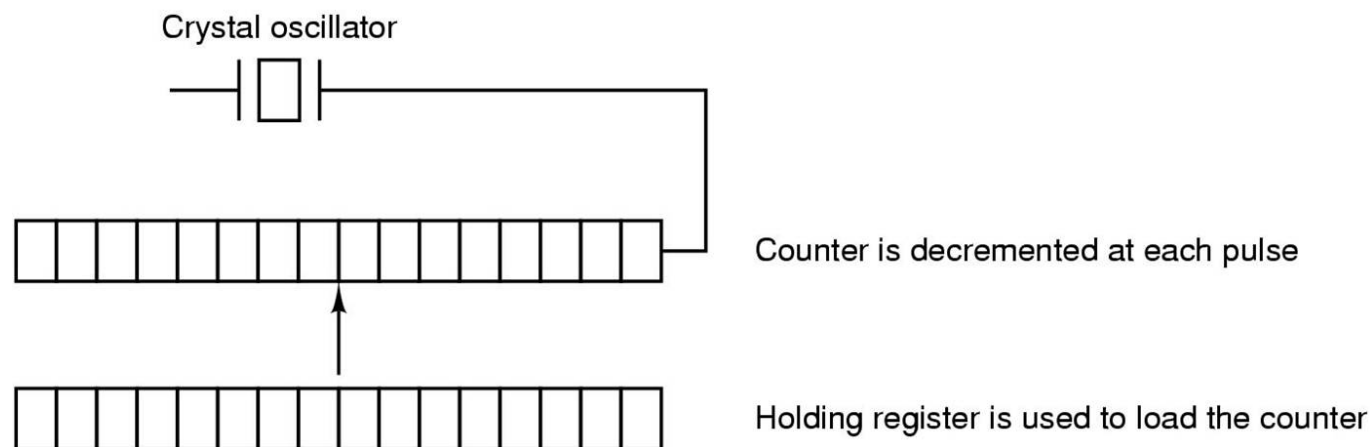
- 时钟(clock), 又称为定时器(timer), 负责维护时间、防止一个进程长期占用CPU等功能
- 时钟既不是块设备, 也不是字符设备

时钟硬件

■ 可编程时钟的组成:

- 晶体振荡器
- 计数器
- 存储寄存器

- 晶体振荡器产生的精确的周期信号；时钟信号送到到计数器，使其递减计数；当计数器变为0时，产生一个CPU中断信号



时钟操作模式

- 一次完成模式(one-shot mode): 当时钟启动时, 它把存储寄存器的值拷贝到计数器中, 然后, 晶体的每一个脉冲使计数器减1。当计数器为0时, 产生一个中断, 并停止工作, 直到软件再一次显式启动它
- 方波模式(square-wave mode): 当计数器为0并产生中断时, 存储寄存器的值自动拷贝到计数器, 这个过程不断地重复下去。周期性的中断称为时钟滴答(clock tick)
- 可编程时钟的优点是其中断频率可由软件控制

时钟软件

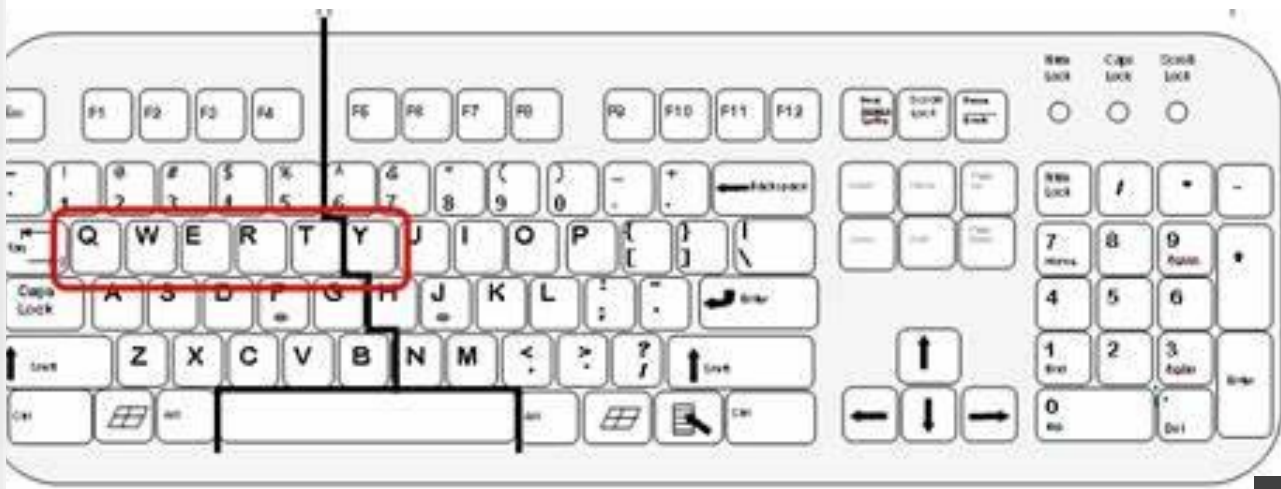
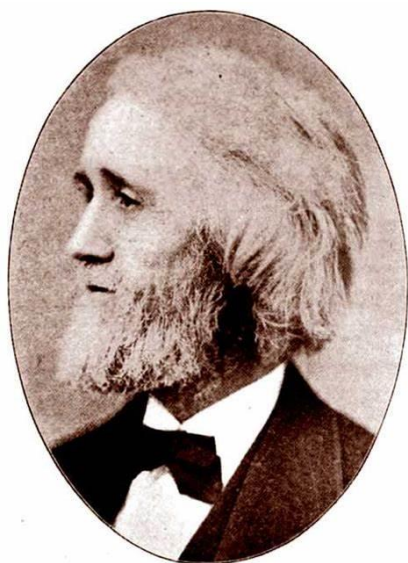
- 时钟硬件所做的工作是每隔一定的时间间隔产生一个中断
- 涉及时间的其他所有工作都必须由软件——时钟驱动程序完成
- 时钟软件的功能
 1. 维护时间
 2. 防止进程超时运行
 3. 对CPU的使用情况记帐
 4. 处理用户进程提出的alarm系统调用
 5. 为系统本身各部分提供监视定时器
 6. 完成监视和统计信息收集

人机交互设备

- 人机交互设备是人和计算机之间建立联系、交流信息的输入输出设备
- 人机交互设备分为输入设备和输出设备
- 输入设备用于把原始数据和处理这些数据的应用程序输入到计算机中，典型的输入设备有键盘、鼠标、扫描仪等
- 输出设备用于将各种计算结果以数字、字符、图形、图像、声音等形式表示出来。典型的输出设备有显示器、打印机等

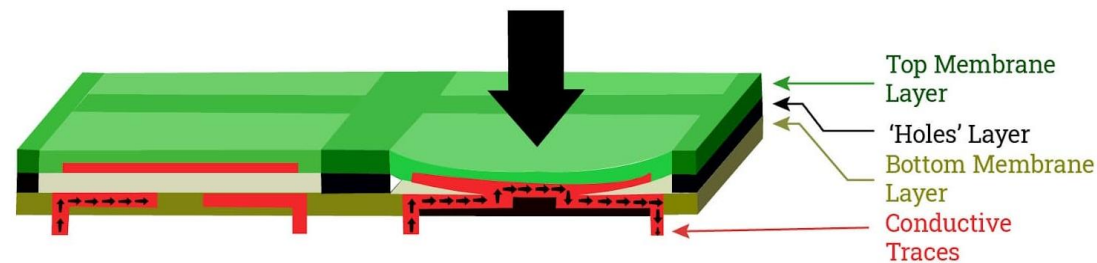
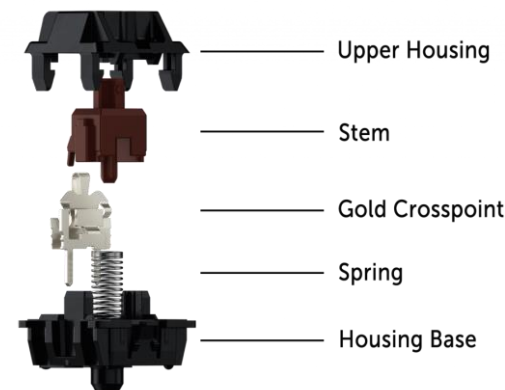
键盘

- 键盘的历史非常悠久，早在1714年就出现了各种形式的打字机，键盘就出现在打字机上
- 1868年，美国人Christopher Latham Sholes（克里斯托夫·拉森·肖尔斯）设计了现代打字机的规范键盘，即QWERTY键盘



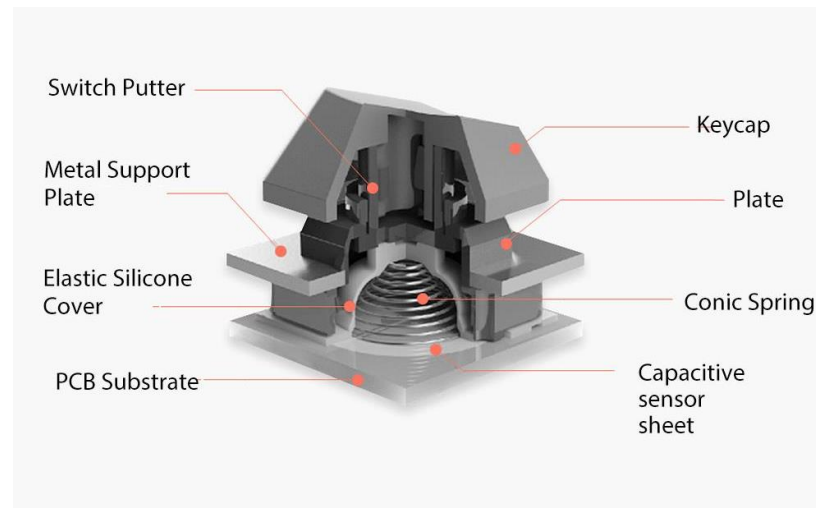
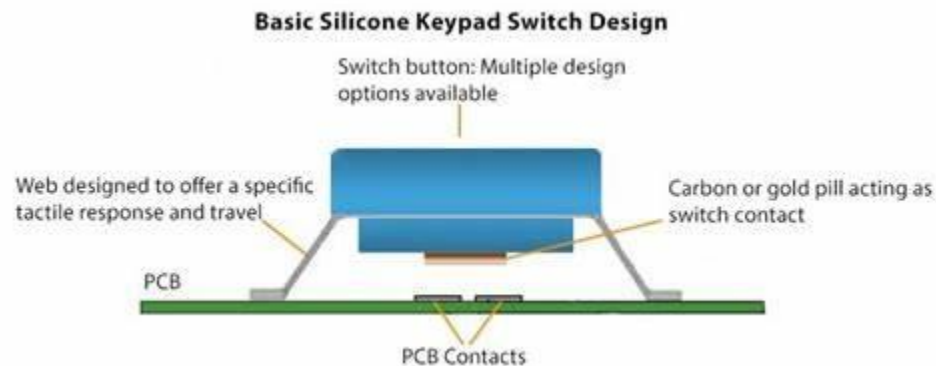
键盘分类——根据按键的接触方式

- **机械式键盘**：键盘底部的PCB板上固定着100多个机械式按键开关，通过触点导通或断开来判断按键是否被触发
- **薄膜式键盘**：由一层按键、三层薄膜及电路板组成。最上层是中心有凸起橡胶垫的按键；三层薄膜中最上层是正级电路，中间是间隔层，下层是负极电路。对应每个按键，上下两层薄膜中都有相应的触点，间隔层有相应的小孔。当按键按下时，上下两层薄膜触点通过小孔而导通



键盘分类——根据按键的接触方式

- **导电橡胶式键盘**：上层是中心有凸起导电橡胶垫的按键，下层是正负极平行交叉的触点，当按键按下时，导电橡胶使其下面触点的正负极接通
- **无接点静电电容式键盘**：按键使用类似电容式开关的原理，通过按键时改变电极间的距离，引起电容容量改变从而获得按键通断信号



键盘的工作原理

- 键盘内部根据按下按键的位置生成位置状态代码（扫描码）传送给计算机，然后由计算机内的软件（键盘驱动程序）把这些位置状态代码转换为相应的按键编码（ASCII字符码）
- 键盘的按键按行（M行）和列（N列）排列，可排 $M \times N$ 个按键，按键识别只需 $M+N$ 条线

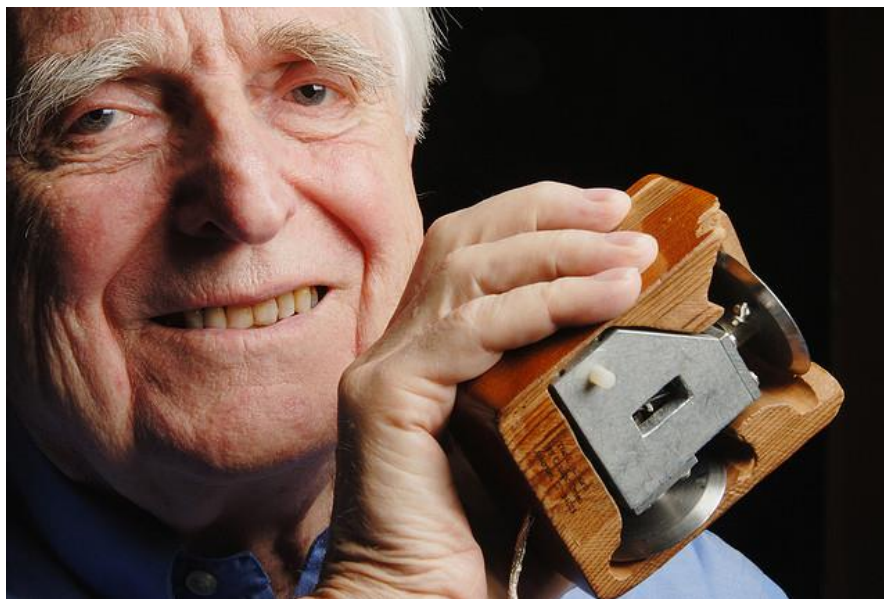


键盘的工作原理

- 键盘内部的控制器采用扫描法来识别按键的位置。如果有闭合键就根据其位置（行值和列值）获得对应的扫描码，当闭合键松开时也对应一个扫描码，前者为闭合扫描码，后者为断开扫描码
- 键盘控制器收到扫描码后，向CPU发出键盘中断请求，由CPU响应键盘中断而调入扫描码，并将其转换为按键编码（ASCII字符码），保存到键盘缓冲区中，以便程序使用

鼠标

- 世界上第一个鼠标1968年诞生于美国斯坦福大学，发明者是Douglas Engelbart（道格拉斯·恩格尔巴特）
- 设计鼠标的初衷是为了快速地屏幕定位，使计算机的操作更加简便

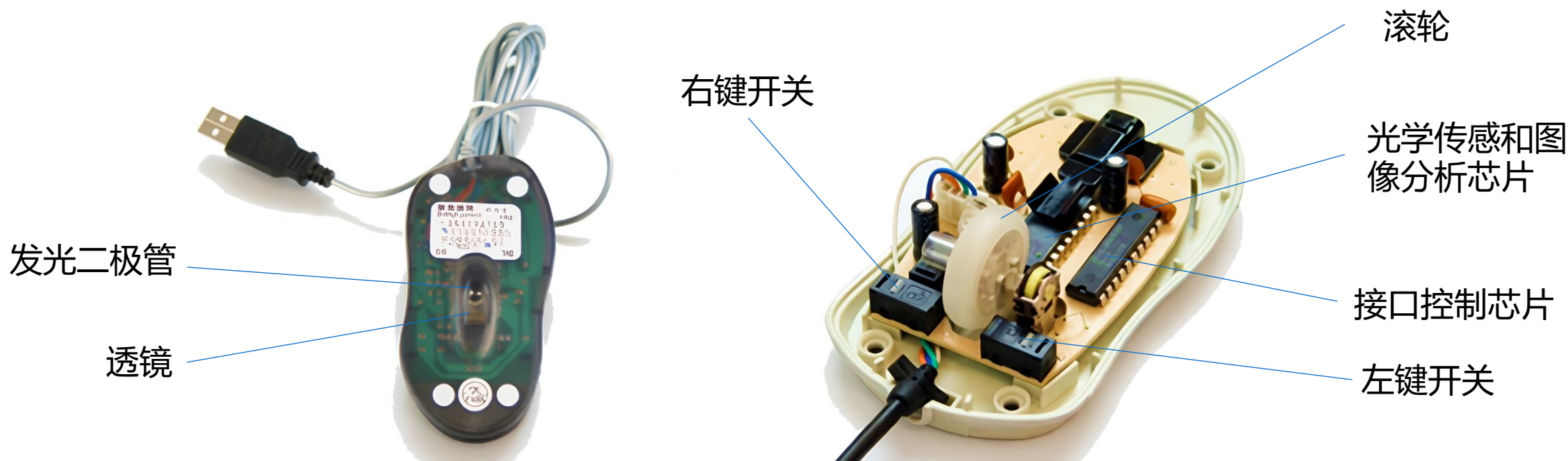


鼠标的分类

- 按键数分类：双键鼠标、三键鼠标和新型的多键鼠标
- 按接口分类：COM、PS/2、USB三类
- 按有无连线分类：有线鼠标和无线鼠标。无线通信方式通常为蓝牙和Wi-Fi
- 按内部结构分类：机械式和光电式

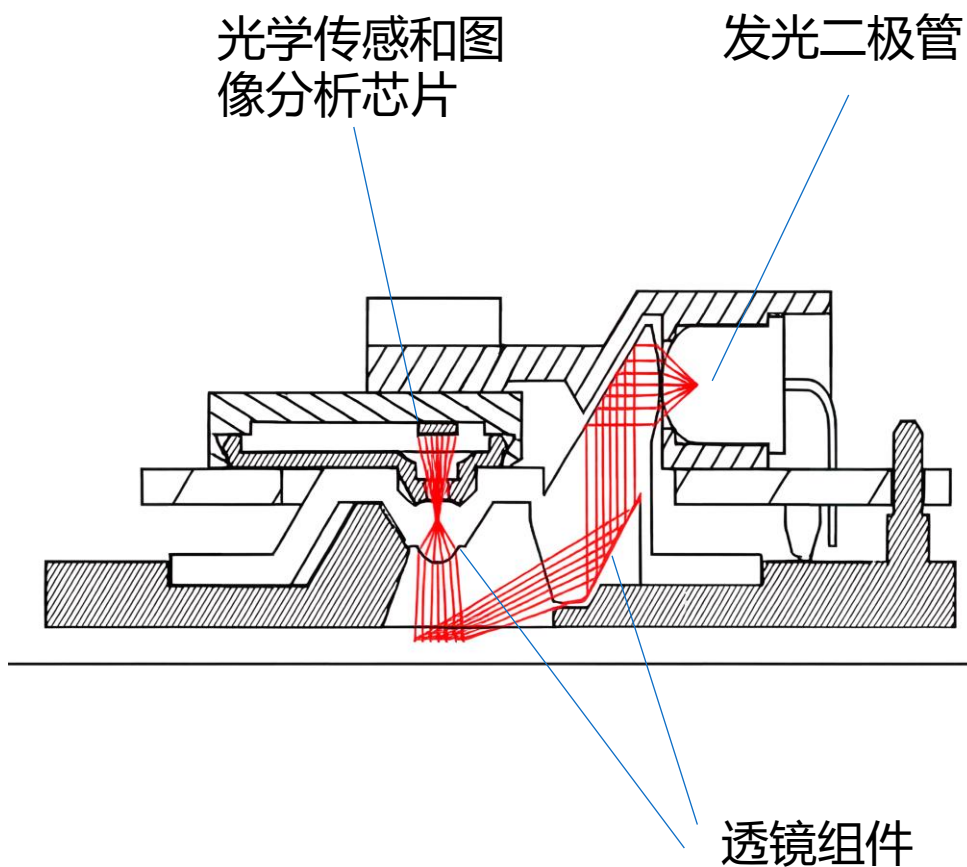
光电式鼠标的工作原理

- 光电鼠标通常由发光二极管、光学透镜、光学感应及图像分析芯片、接口控制芯片、轻触式按键、滚轮、外壳等部分组成



光电式鼠标的工作原理

- 光电鼠标通过发光二极管发出光线照亮鼠标底部表面，然后将光电鼠标底部表面反射回的一部分光线经过一组光学透镜传输到一个光学传感器件内成像
- 当鼠标移动时，其移动轨迹会被记录为一组高速拍摄的连续图像。然后利用专用图像分析芯片对其进行处理，通过对这些图像上特征点位置的变化进行分析，给出鼠标的移动方向和移动距离，最后将这些信息传输给接口控制芯片，接口控制芯片通过接口向主机传送鼠标移动的信息，主机再通过处理使屏幕上的光标与鼠标同步移动



显卡与显示器

- 显卡又称为显示适配器，工作在主机与显示器之间。负责把CPU送来的图像数据处理成显示器接收的格式，再送到显示器形成图像
- 显卡和显示器相互连接协同工作

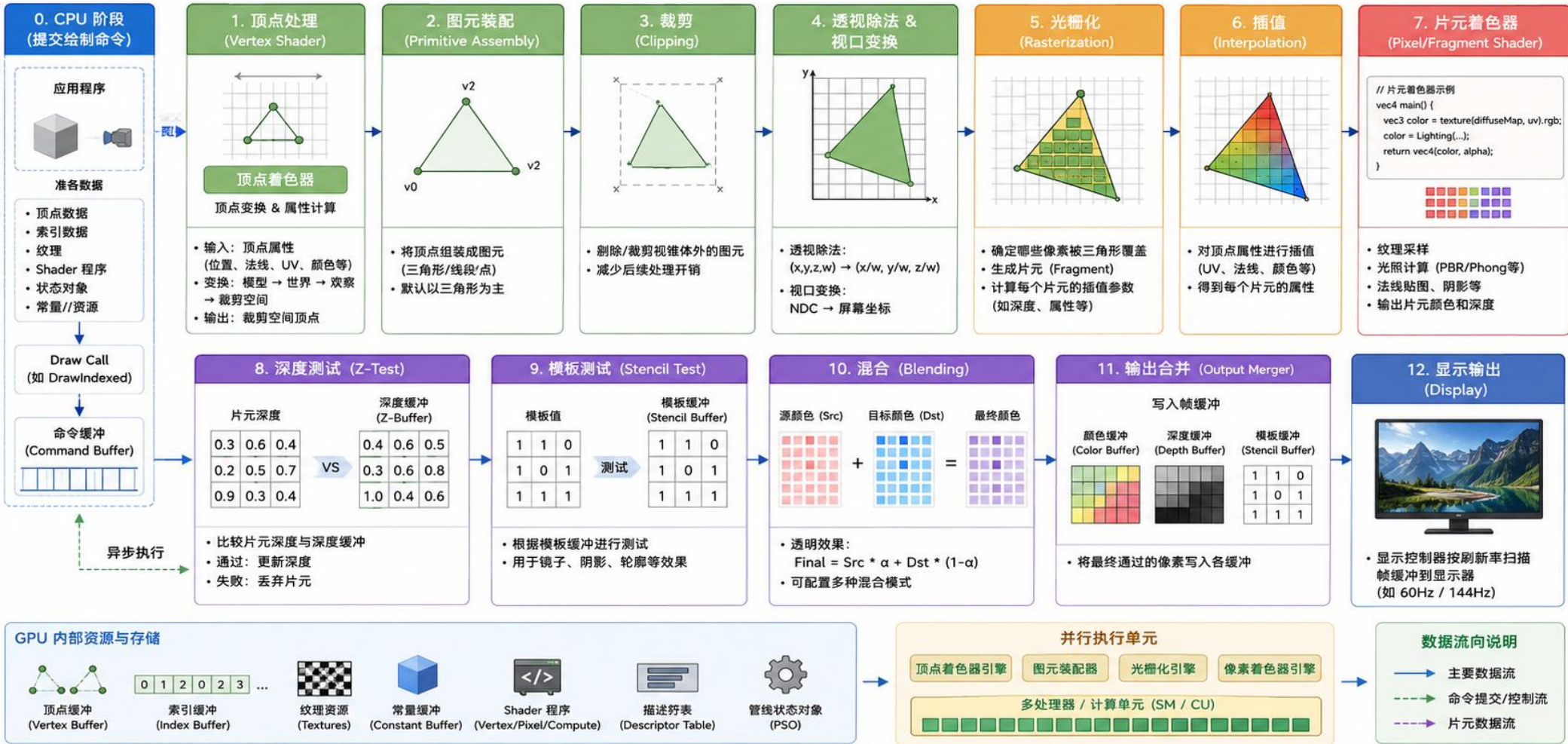


显卡的工作过程

- 首先，由CPU将需要处理的数据通过AGP(Accelerated Graphics Port)或PCIe(PCI Express)总线传送到显卡的显存中，显示芯片（GPU）提取存储在显存中的数据进行处理
- GPU内部由管线（pipeline）构成，分为顶点管线和像素管线，顶点管线主要负责3D建模，像素管线主要负责3D渲染
- 当GPU处理完毕后，数字图像数据会被送入随机存储数字模拟转换器（RAMDAC）转换成显示器需要的模拟数据（数字信号接口不需此转换），由显示输出接口传输给显示器，成为人们所看到的图像

显卡的工作过程

GPU 图形管线处理流程（从顶点到像素）



显卡输出接口类型

- 主流显卡输出接口包括VGA接口、 DVI接口、 HDMI接口和DisplayPort接口
 - VGA——Video Graphics Array (视频图形阵列)
 - DVI——Digital Visual Interface (数字视频接口)
 - HDMI——High Definition Multimedia Interface (高清多媒体接口)
 - DisplayPort——通常简称为DP



DisplayPort

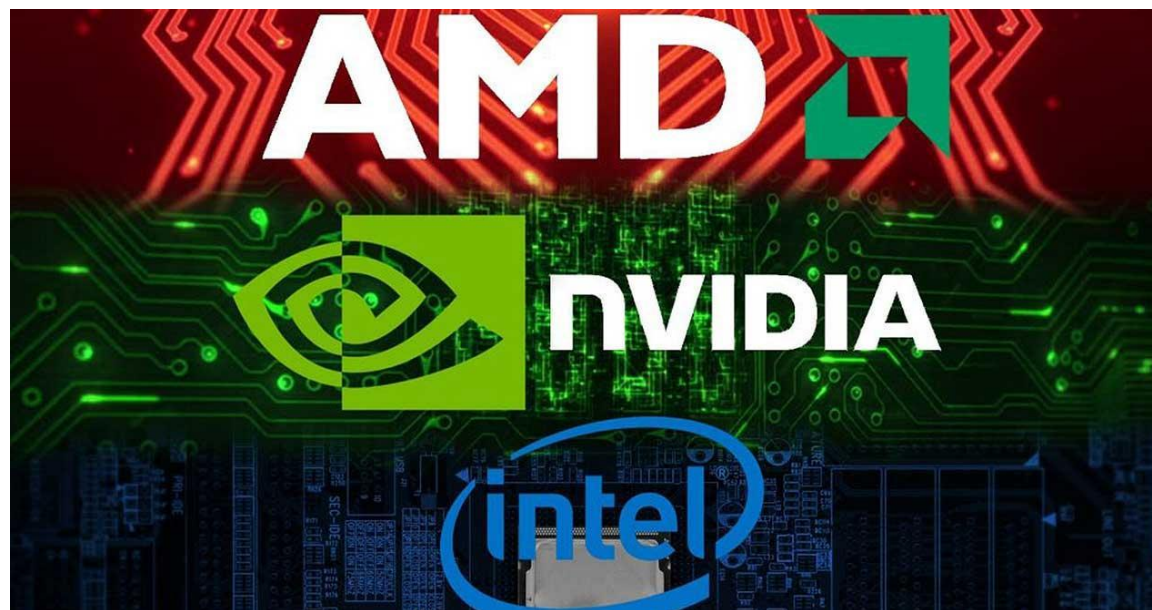
HDMI

VGA

DVI

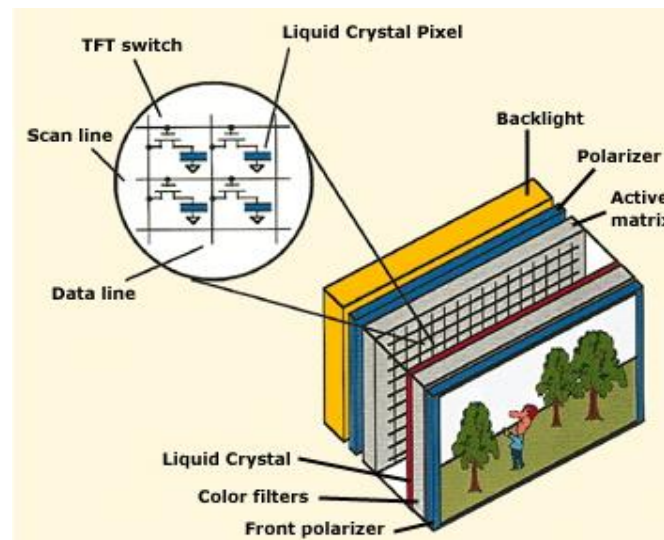
主流显示芯片

- 显卡的品牌众多，而设计和制造显示芯片的主流厂家仅有NVIDIA、Intel和AMD等几家



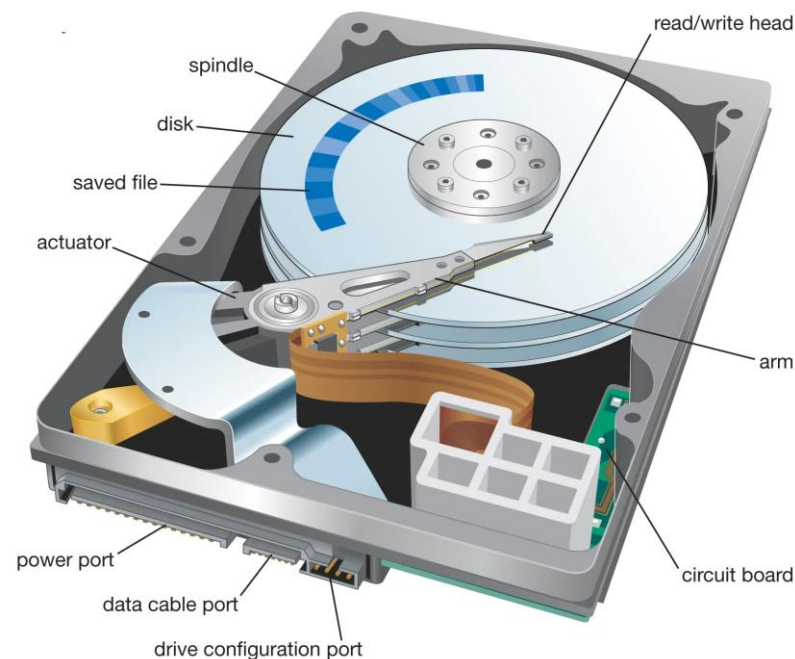
显示器

- 显示器是计算机最重要的输出设备。目前使用的显示器主要为LCD（液晶）显示器，早期的CRT（阴极射线管）显示器已基本被淘汰
- LCD显示器利用液晶的光电效应，即液晶分子的排列在电场作用下发生变化，影响其液晶单元的透光率或反射率，从而影响它的光学性质，产生具有不同灰度层次及颜色的图像



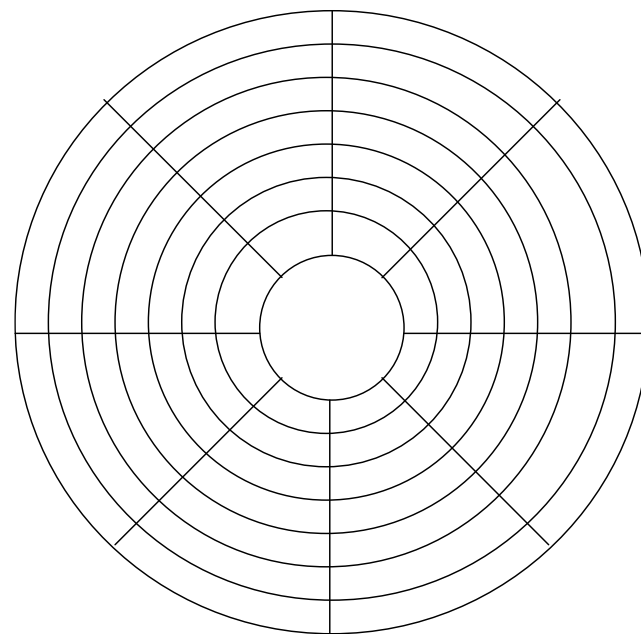
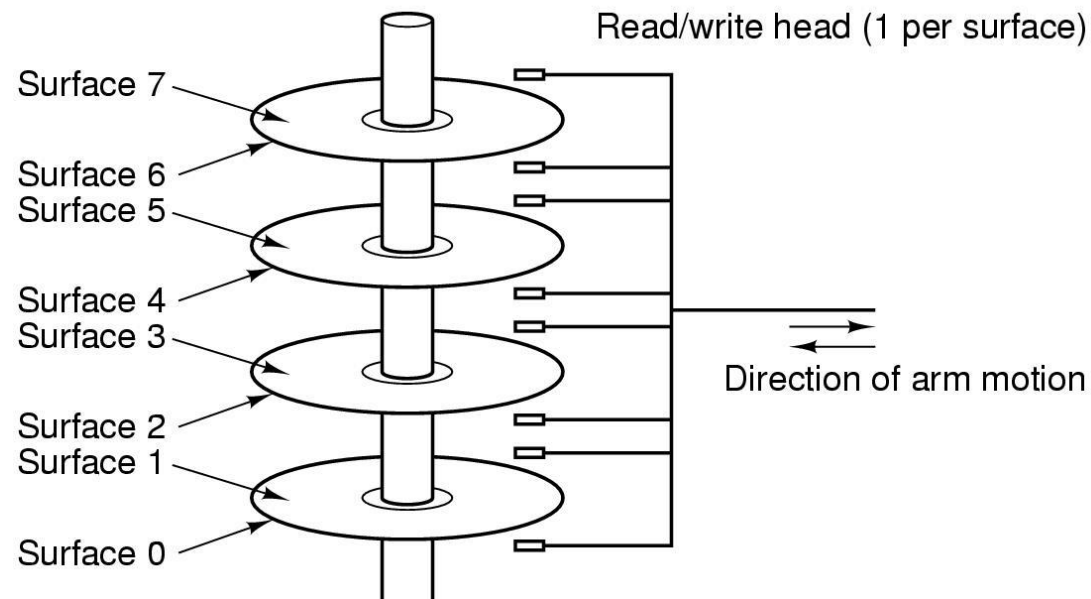
硬盘 (HDD)

- 磁盘包括硬盘和软盘，是通过磁性介质存储数据的设备
- 硬盘是磁盘的一种具体形式，是由覆盖有铁磁性材料的金属或玻璃盘片、磁头及密封外壳组成的固定存储设备，具有大容量和较高的读写速度
- 尽管硬盘驱动器（Hard Disk Drive, HDD）和硬盘（Hard Disk）是两个概念，但是由于两者通常被封装在一起，所以无论是硬盘还是硬盘驱动器通常都是指二者封装在一起所形成的设备



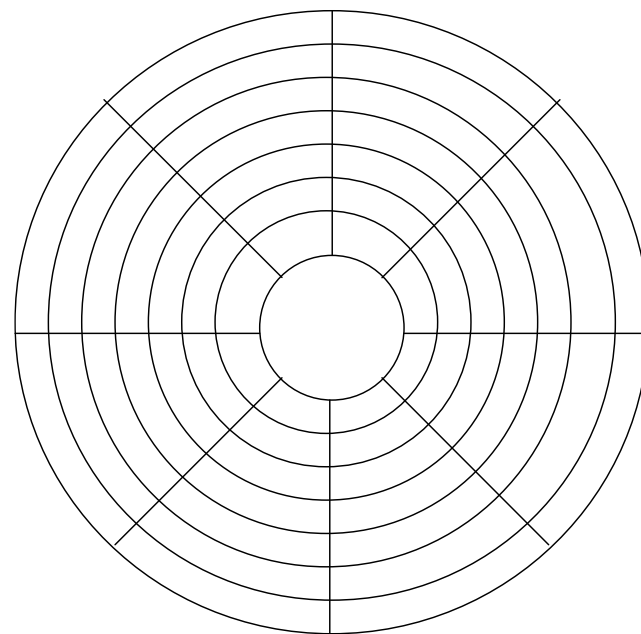
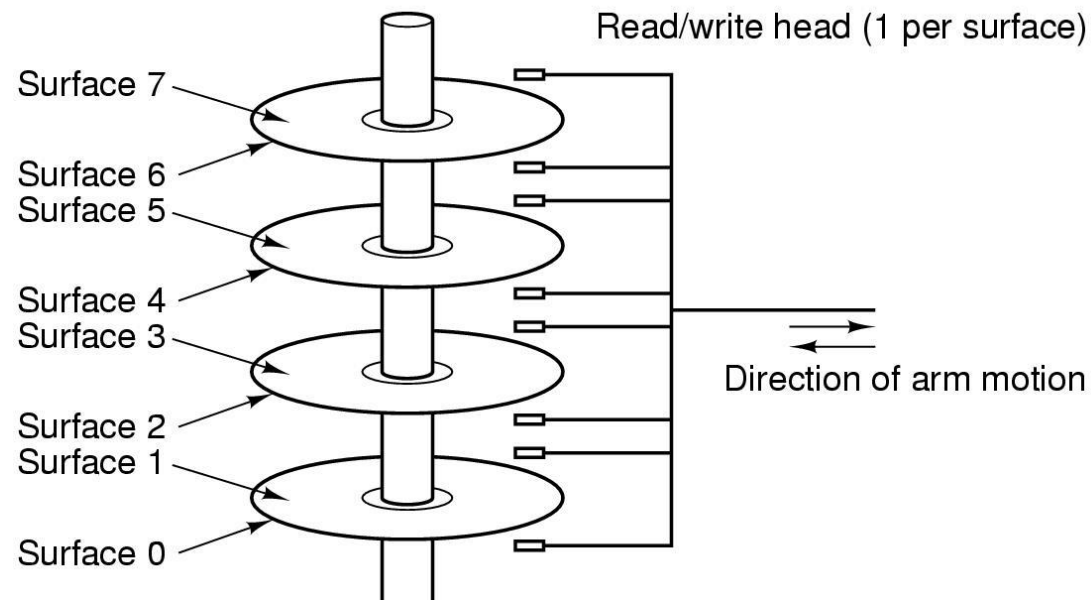
硬盘的结构

- 硬盘由许多盘片组成，每个盘片有两个盘面
- 每个盘面上有一个读写磁头，盘面号即磁头号，所有磁头在磁头臂的作用下同时移动。磁头从0开始编号
- 每个盘面被划分成许多同心圆，这些同心圆称为磁道，磁道从外向内从0开始顺序编号
- 磁道被分成许多扇区，每个扇区包含相同的字节数，典型值是512字节。扇区从 1 开始编号
- 所有盘面上的同一磁道构成一个圆柱，称作柱面



硬盘的结构

- 硬盘操作按柱面优先，而不是按盘面优先
- 写入数据时，首先确定柱面，即所有磁头一起定位到指定柱面。当前柱面的当前磁道写满后，再在当前柱面的下一个磁道写入，只有当前柱面全部写满后，才能将磁头移动到下一个柱面
- 对硬盘进行分区时，各个分区也是以柱面为单位划分，例如即从0柱面到1023柱面，从1024柱面到1279柱面等等，不存在一个柱面属于多个分区



磁盘寻址——CHS

- **CHS**，即 Cylinder(柱面)、Heads(磁头)、Sector(扇区)，使用这三个参数构成的三元组(C, H, S)来定位一个扇区
- 柱面参数确定磁头臂移动到某个磁道，磁头参数确定是哪个盘面上的磁道，扇区参数确定该磁道上某个扇区的具体位置

磁盘寻址——LBA

- **LBA**——Logical Block Addressing, 逻辑块寻址
- 将 (C, H, S) 三元组转换为一维的线性地址, 即 0柱面0磁头1扇区编址为逻辑0扇区, 0柱面0磁头2扇区编址为逻辑1扇区,

磁盘寻址

■ CHS → LBA转换

$$LBA = (C \times HPC + H) \times SPT + S - 1$$

■ LBA → CHS转换

$$\begin{cases} C = LBA / (HPC \times SPT) \\ H = (LBA / SPT) \% HPC \\ S = LBA \% SPT + 1 \end{cases}$$

- 其中，HPC为每柱面磁头数(heads per cylinder)，SPT为每磁道扇区数(sectors per track)，/为整数除法，%为取余

磁盘I/O访问时间

- **寻道时间 T_s** : 磁头移动到指定柱面的机械运动时间, 通常每个柱面移动需要10ms左右
- **旋转延迟时间 T_r** : 等待指定扇区旋转到磁头下的机械运动时间。它与磁盘转速相关, 例如某硬盘转速为7200rpm, 则 $T_r=4.17\text{ms}$
- **实际数据传输时间 T_t** : 数据读写的时间, 与读写的字节数和磁盘转速有关

磁盘I/O访问时间

- 合理组成磁盘数据上的存储位置可提高磁盘I/O性能

- 例：某磁盘平均柱面寻道时间为10ms，磁盘转速为10000rpm，每个磁道有32个扇区，每个扇区512字节。现读取包含256个扇区的文件，试估算磁盘I/O访问时间

1. 文件采用顺序存储： $10 + (3 + 6) * 8 = 82\text{ms}$

2. 文件由256个随机分布的扇区构成： $(10 + 3 + 6/32) * 256 = 3376$

- 随机存储的访问时间为顺序存储的41倍

磁盘I/O访问时间

优化驱动器

你可以优化驱动器以帮助计算机更高效地运行，或者分析驱动器以了解是否需要对其进行优化。

状态(T)

驱动器	媒体类型	上次分析或...	当前状态
System (C:)	硬盘驱动器	2025/5/13 ...	正常(碎片整理已完成 0%)
Teaching (D:)	硬盘驱动器	2025/5/13 ...	正常(碎片整理已完成 0%)
Research (E:)	硬盘驱动器	2025/5/13 ...	正常(碎片整理已完成 0%)
Working (F:)	硬盘驱动器	2025/5/13 ...	正常(碎片整理已完成 0%)

分析(A)

优化(O)

已计划的优化

启用

更改设置(S)

正在按计划的节奏分析驱动器，并根据需要对其进行优化。

频率: 每周

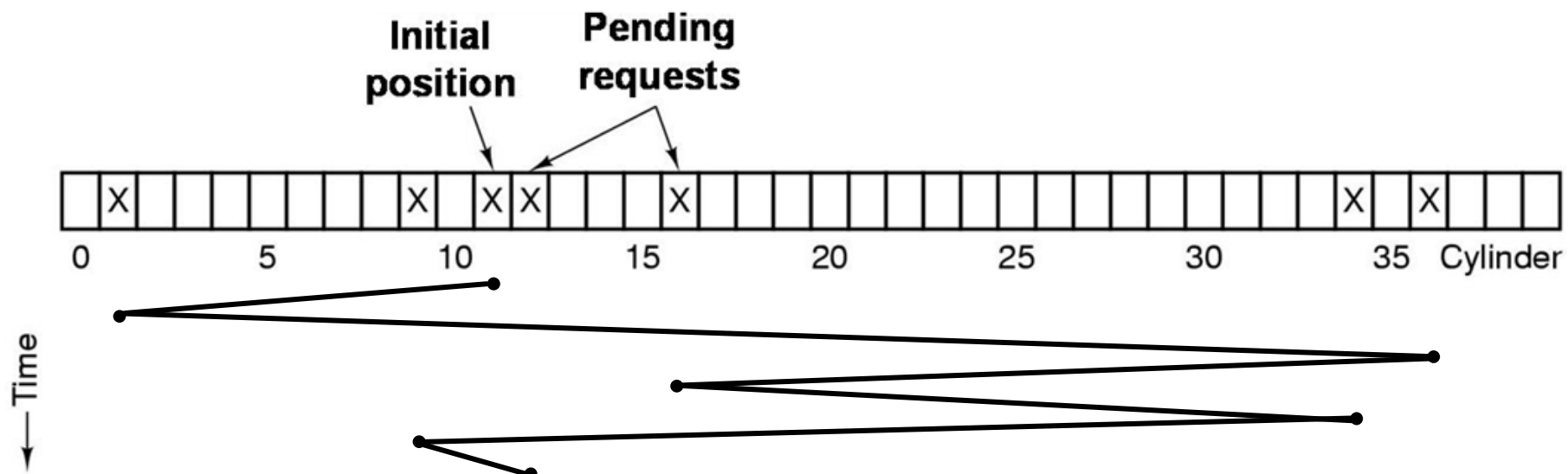
关闭(C)

磁盘臂调度算法

- 来自不同进程的磁盘I/O请求构成一个随机分布的请求队列
- 由于寻道时间在访问时间中占主要部分，磁盘臂调度的主要目标就是减少请求队列对应的平均寻道时间

先来先服务(FCFS)

- 磁盘I/O执行顺序为磁盘I/O请求的先后顺序
- 该算法的特点是公平性，但是性能比较差
- **例**：一个具有40个柱面的磁盘，假设正在对柱面11进行寻道时，又按顺序到来对柱面1、36、16、34、9和12的请求。使用FCFS算法，总共要移动111个柱面

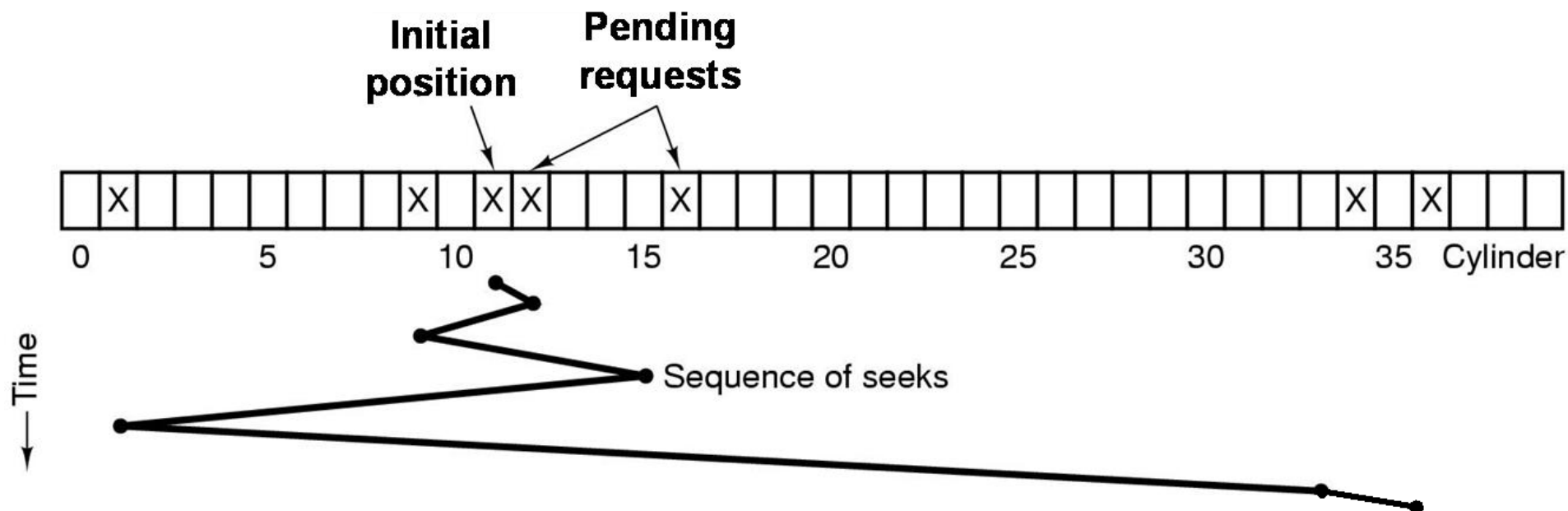


最短寻道优先(SSF, Shortest Seek First)

- 考虑磁盘I/O请求队列中各请求的磁头定位位置，选择从当前磁头位置出发，移动最少的磁盘I/O请求
- 该算法的目标是使每次磁头移动时间最少
- 对中间的磁道有利，可能会有进程处于饥饿状态

最短寻道优先(SSF, Shortest Seek First)

- 例，当前位置11，寻道序列1、36、16、34、9和12
- 使用SSF算法，总共需要移动61个柱面

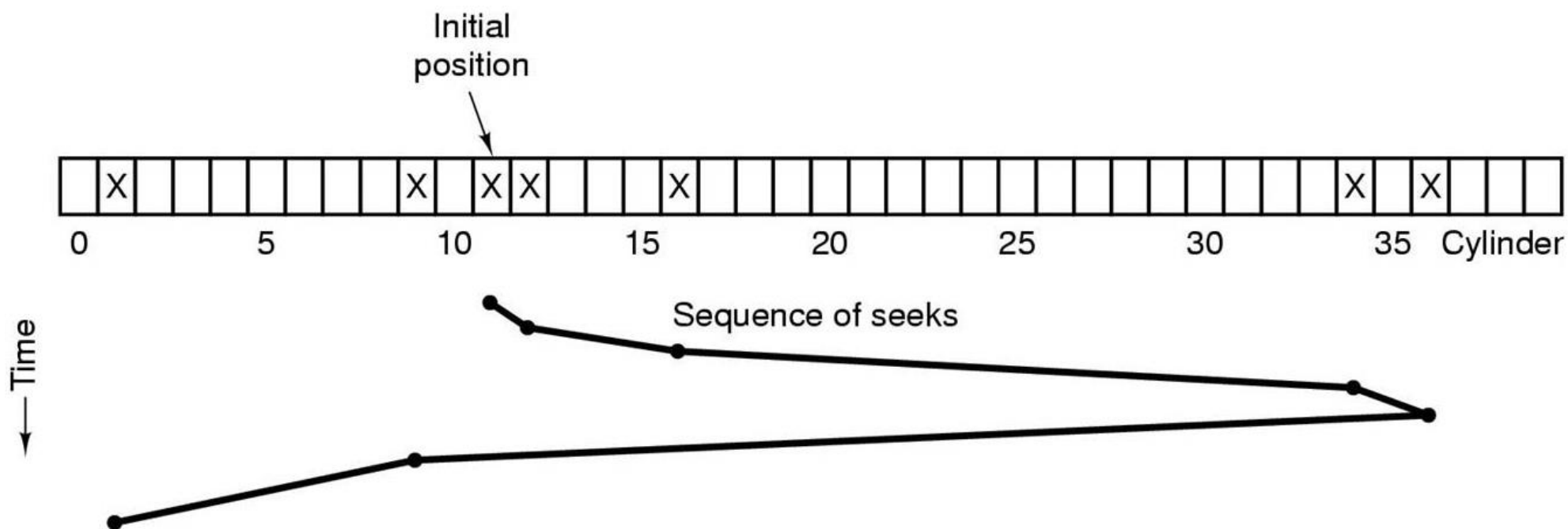


电梯算法

- 选择在磁头前进方向上从当前位置移动最少的磁盘I/O请求执行，没有前进方向上的请求时才改变方向
- 该算法是对SSF算法的改进，磁盘I/O较好，且没有进程会饿死
- 对中间磁道有利

电梯算法

- 例，当前位置11，寻道序列1、36、16、34、9和12
- 使用电梯算法，总共需要移动60个柱面

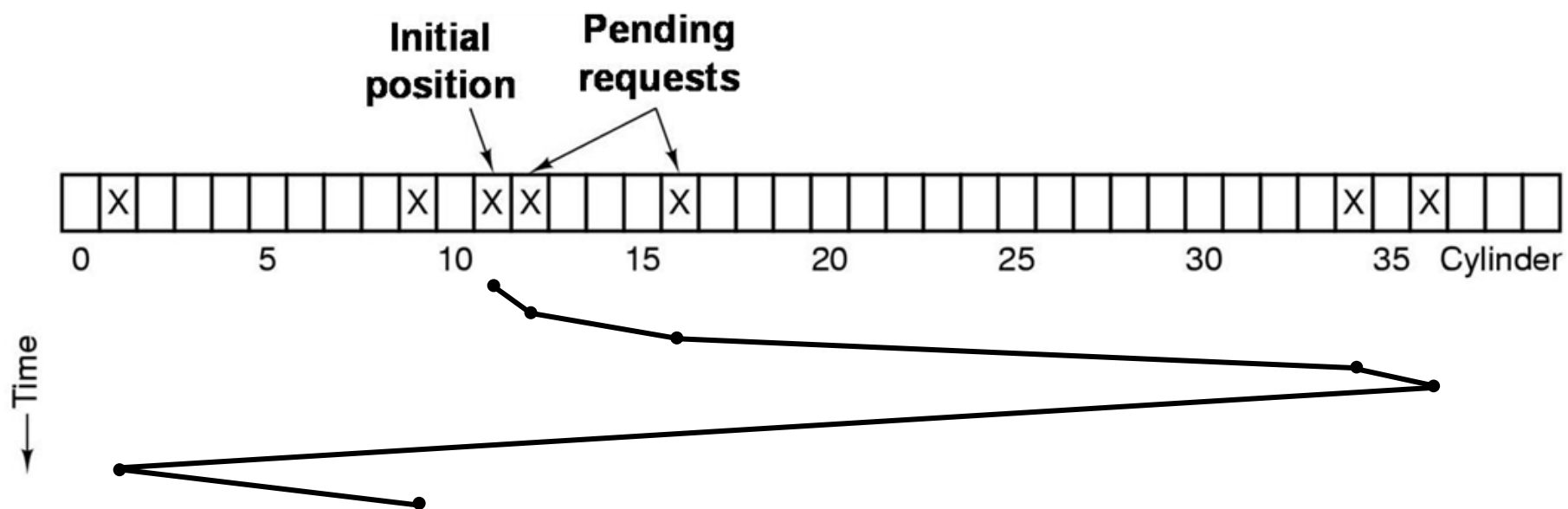


单向扫描算法

- 该算法可改进电梯算法对中间磁道的偏好
- 在一个方向上使用电梯算法，当到达边沿时直接移动到另一沿的第一个位置
- 实验表明，该算法在中负载或重负载时，磁盘I/O性能比电梯算法好

单向扫描算法

- 例，当前位置11，寻道序列1、36、16、34、9和12
- 使用单向扫描算法算法，总共需要移动68个柱面



SSD

- SSD是固态驱动器（Solid State Drive）的缩写，通常称为固态硬盘
- SSD三个核心组件为NAND闪存、控制器及固件，NAND闪存负责数据的存储，控制器与固件协同合作完成数据的读写，维护SSD的性能和延长使用寿命
- SSD一般会带有缓存芯片



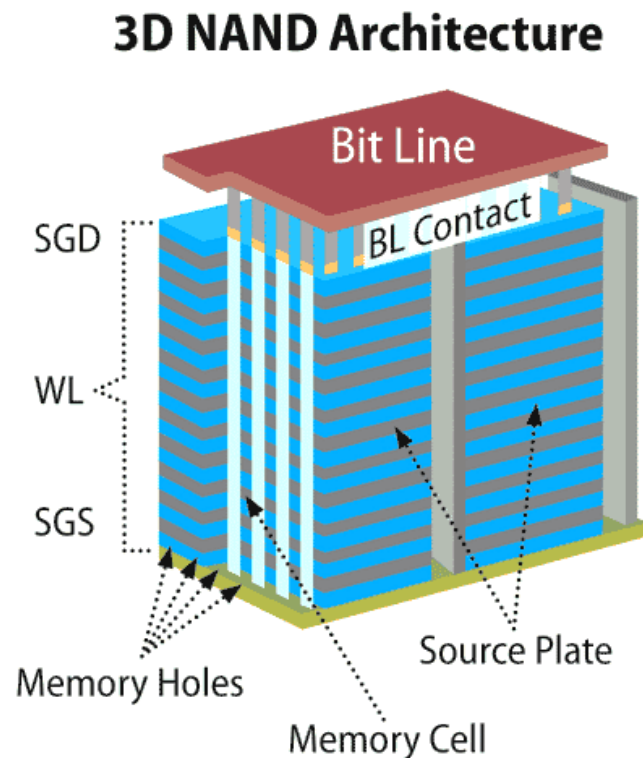
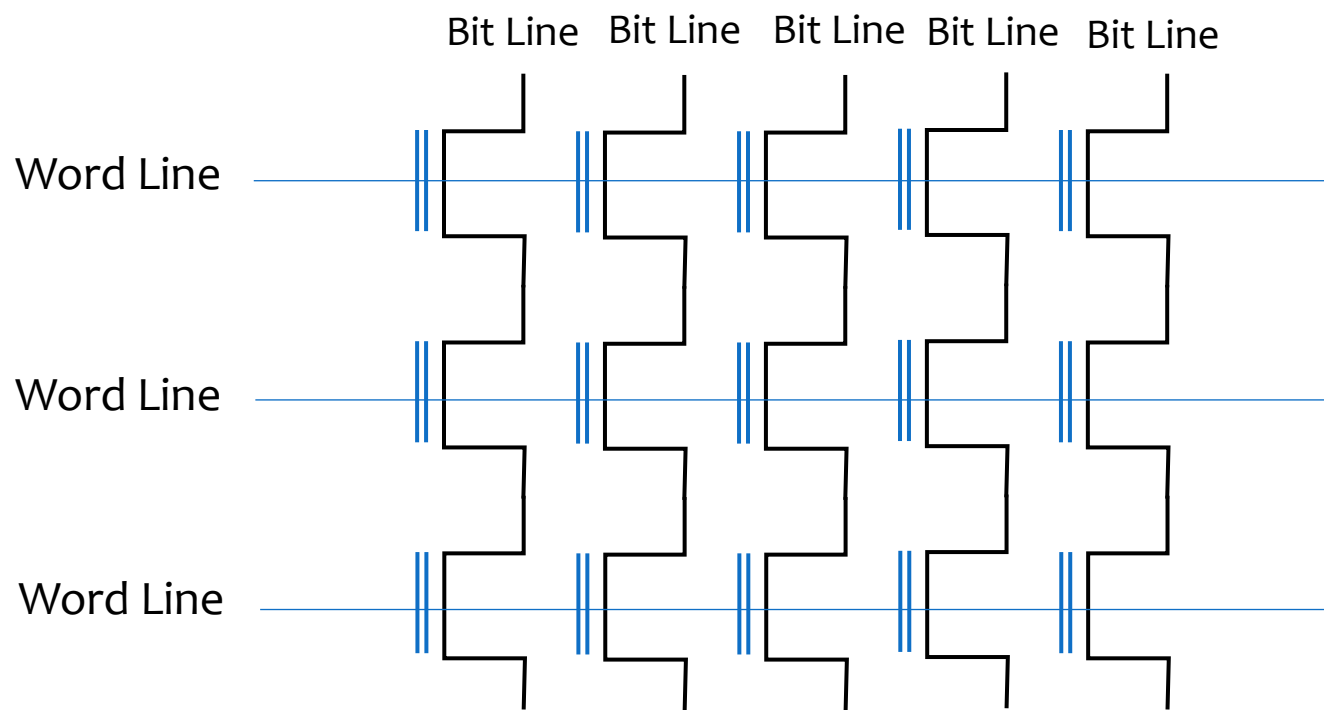
SSD

- SSD的存储核心是NAND Flash，NAND Flash的基本单元是浮栅晶体管
- 浮栅晶体管由控制栅极、氧化层、浮栅极、源极和漏极构成，其中浮栅极可以存储电子，上下被氧化层包围，避免电子流出
- 浮栅极如果其中没有电子，表示状态1；存储电子后表示状态0
- 浮栅晶体管写操作只能将1改写成0，不能将0改为1。将0改为1，需要采取另外一个称为擦写的操作，即将浮栅极中的电子拉出，擦写操作会损伤隧道氧化层，所以SSD有擦写次数的限制



SSD

- NAND Flash是由大量的浮栅晶体管构成阵列。传统的NAND Flash以二维的方式组成，随着闪存技术制造工艺的发展，3D NAND Flash成为主流技术



SSD的读写操作

- NAND Flash组成成多个Block，每个Block包含多个Page，通常地每个Page的大小为4k或者8k
- NAND的存取必须以Page为单位，即每次读写至少是一个Page
- NAND只能读或写单个Page，但不能覆盖写某个Page，覆盖写入时必须先擦写，再写入。由于擦写的电压较高，所以必须以Block为单位。因此，没有空闲的Page时，必须要找到没有有效内容的Block，先擦写，然后再选择空闲的Page写入
- 在SSD中，一般会维护一个映射表，维护逻辑地址到物理地址的映射。每次读写时，可以通过逻辑地址直接查表计算出物理地址，与传统的机械磁盘相比，省去了寻道时间和旋转时间

SSD比HDD的比较

■ SSD的优势

- 定位数据快：HDD需要经过寻道和旋转，才能定位到要读写的数据块，而SSD通过映射表直接计算即可
- 读写速度快：HDD的速度取决于旋转速度，SSD的读写速度远超HDD，例如SSD可达500MB/s以上，而HDD通常低于200MB/s

■ SSD的劣势

- 成本高：SSD价格约是HDD的2~3倍
- 寿命限制：闪存存在擦写次数限制
- 数据恢复困难：HDD损坏后可通过物理手段恢复数据，成功率较高；SSD损坏后难以修复

廉价磁盘冗余阵列

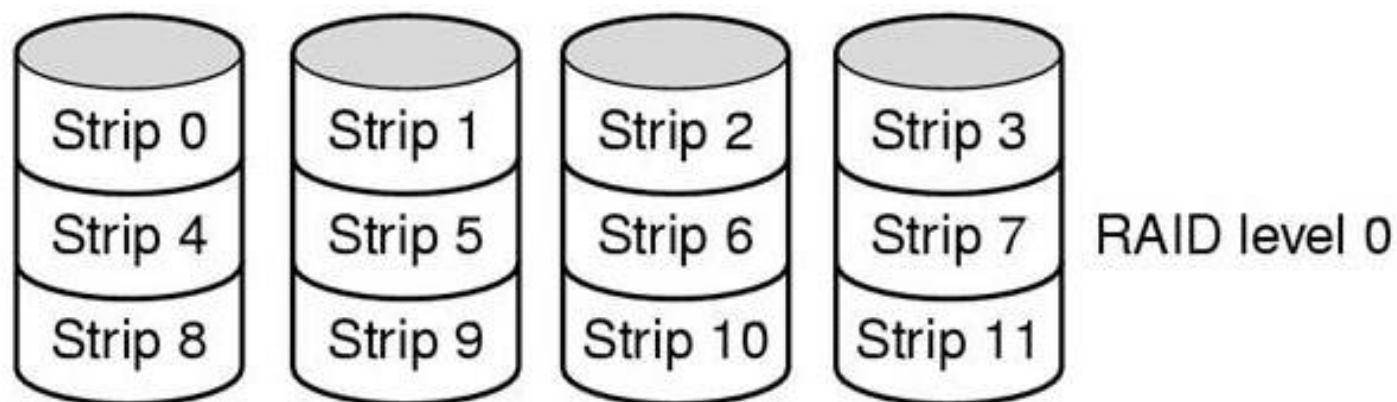
- 廉价磁盘冗余阵列RAID(Redundant Array of Independent Disk) 是利用一台磁盘阵列控制器，统一管理和控制一组磁盘驱动器，组成一个速度快、可靠性高、性能价格比好的大容量磁盘系统
- RAID填补了CPU速度快与磁盘设备速度慢之间的间隙
- **策略**：用一组较小容量的、独立的、可并行工作的磁盘，代替一个大容量的磁盘，应用冗余技术，数据能多方式组织和分布存储，提高了I/O性能和系统可靠性

RAID的特性

- RAID的构成： RAID是一组物理磁盘驱动器，可被操作系统看作是单一逻辑磁盘驱动器
- 条带化(striping)： 将连续的数据分成条带(stripe)分布式存储到不同的物理磁盘驱动器上，每个条带由k个扇区组成
- 数据的读写操作在多个硬盘上并行执行，因此成倍地提高读写速度
- 冗余磁盘： 其作用是保存奇偶校验信息，当磁盘出现失误时它能确保数据的恢复。
- RAID共有7级组成，RAID0 ~ RAID6，其中RAID0无校验； RAID1 ~ RAID6的区别仅是实现冗余校验的方法不同

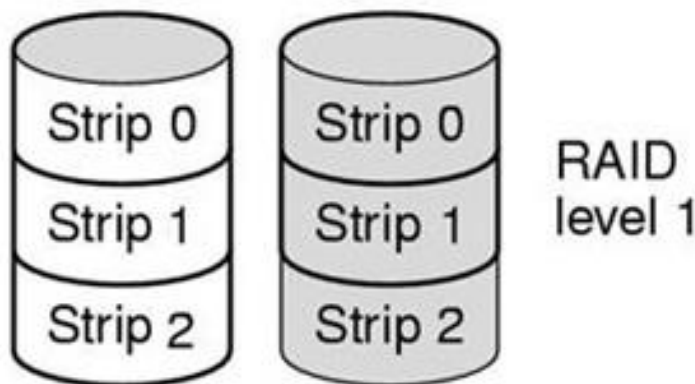
RAID0

- RAID0至少需要两块或以上大小相同的硬盘，将连续的条带(stripe)以轮转方式写到全部驱动器上
- 读写速度快
- 不具有容错性



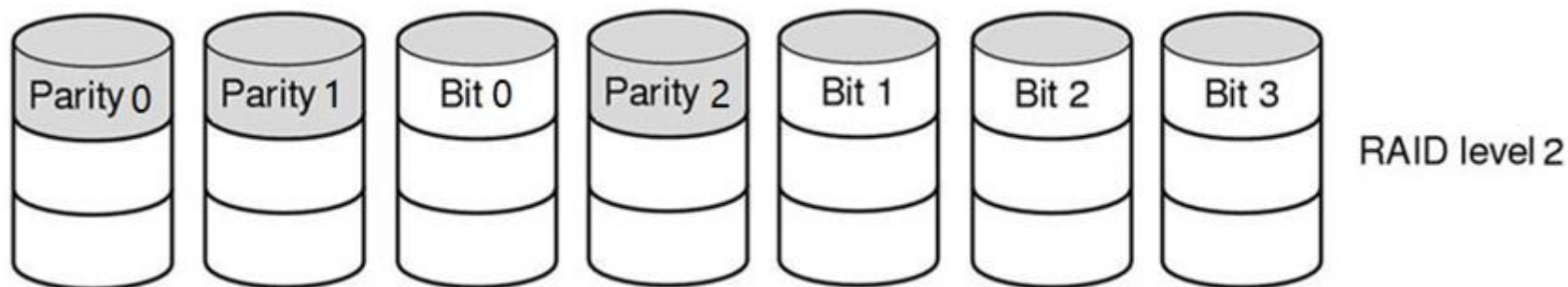
RAID1

- RAID1两块硬盘互为镜像
- 容错性：一个驱动器出现故障，数据可以从镜像驱动器获得
- 实际容量等于一块硬盘的容量，写操作速度与单块硬盘相同，读操作时，可以使用其中任意副本，因此速度可以是单块硬盘的2倍



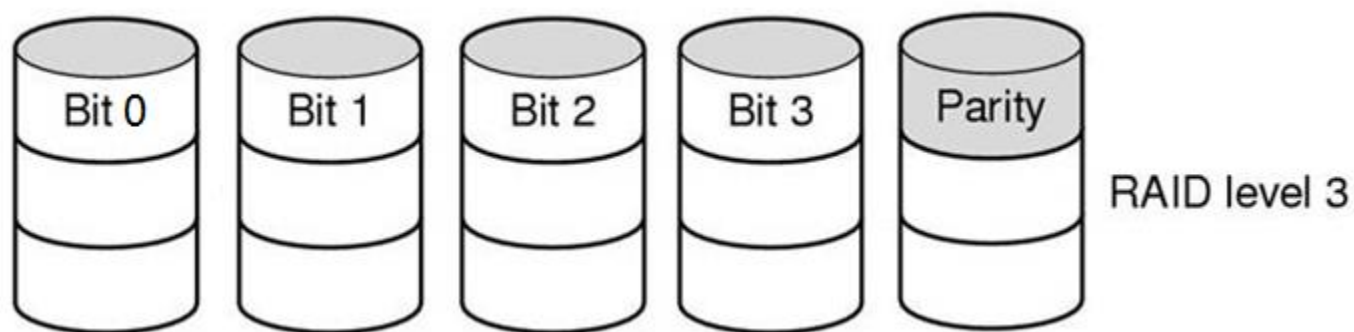
RAID2

- RAID2工作在字或字节的基础上
- 例如，每个字节分割成4位的半字节，对每个半字节加入一个汉明码形成7位的字，存放在7个驱动器上
- 驱动器的磁盘臂同步工作，每个磁盘的磁头都在相同位置



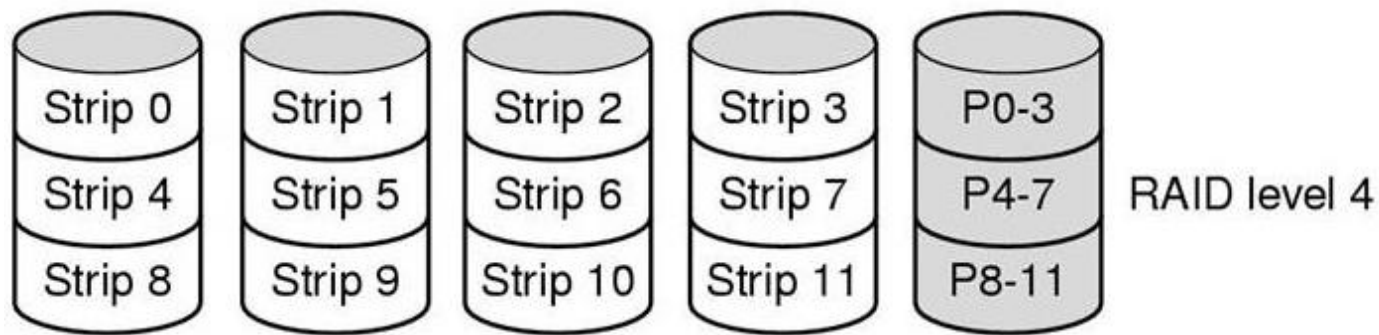
RAID3

- RAID2的简化版本
- 为每个数据字计算一个奇偶检验位并将其写入一个奇偶驱动器中
- 容错：如果一个驱动器崩溃，控制器只需假设该驱动器的位为0，如果有奇偶错误，则原来的位为1



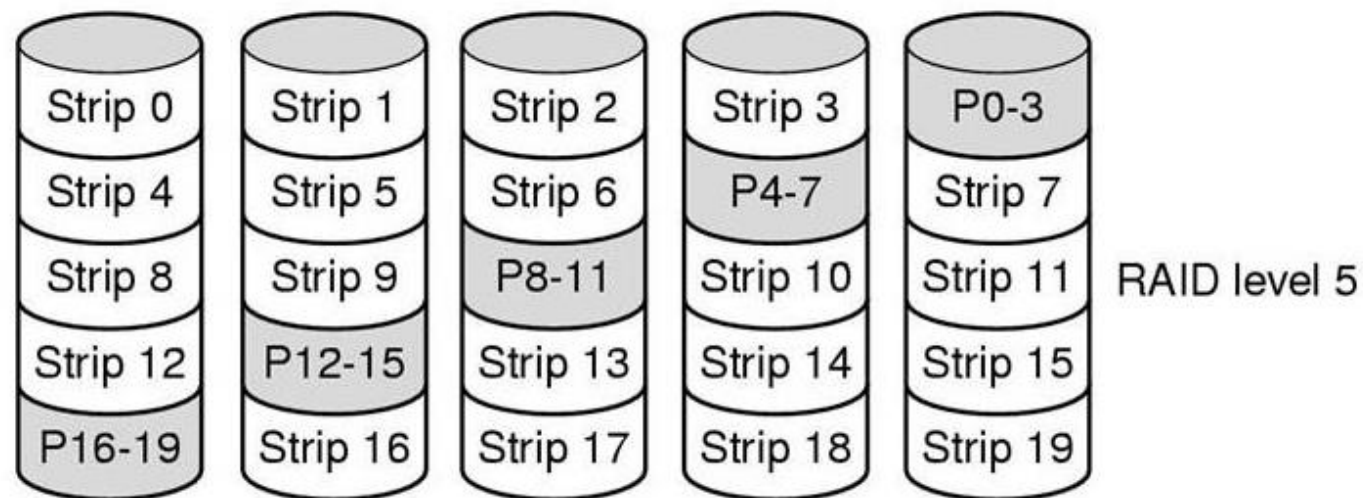
RAID4

- 与RAID0相似，但是对各个条带计算出一个奇偶条带，并将其写入一个额外的驱动器中
- 容错：如果一个驱动器崩溃，损失的字节可以从奇偶驱动器计算出来
- 缺点：奇偶驱动器成为瓶颈



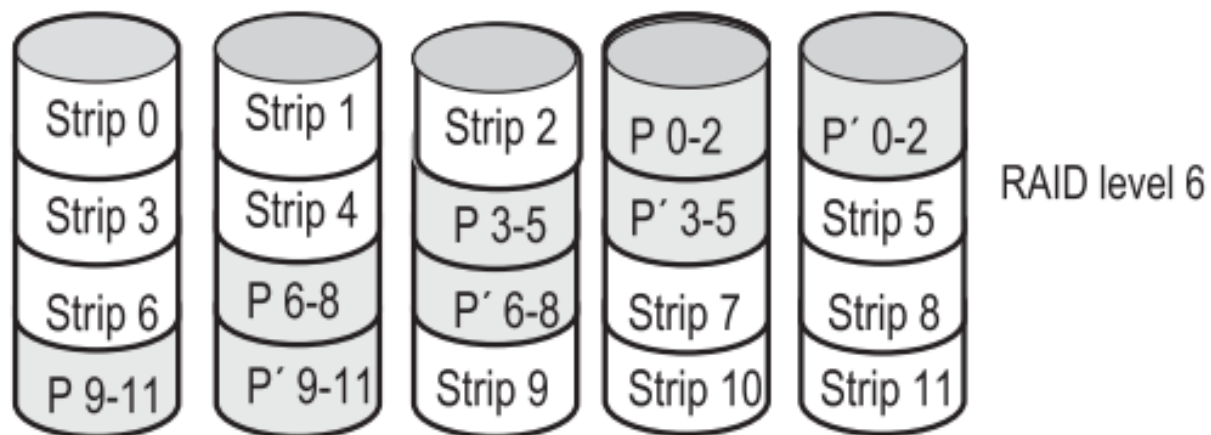
RAID5

- 以循环方式在所有驱动器上均匀地分布奇偶检验位
- 组建RAID 5 要求至少3块硬盘，可以允许坏掉1块硬盘
- 缺点：如果一个驱动器崩溃，重构故障驱动器的内容非常复杂



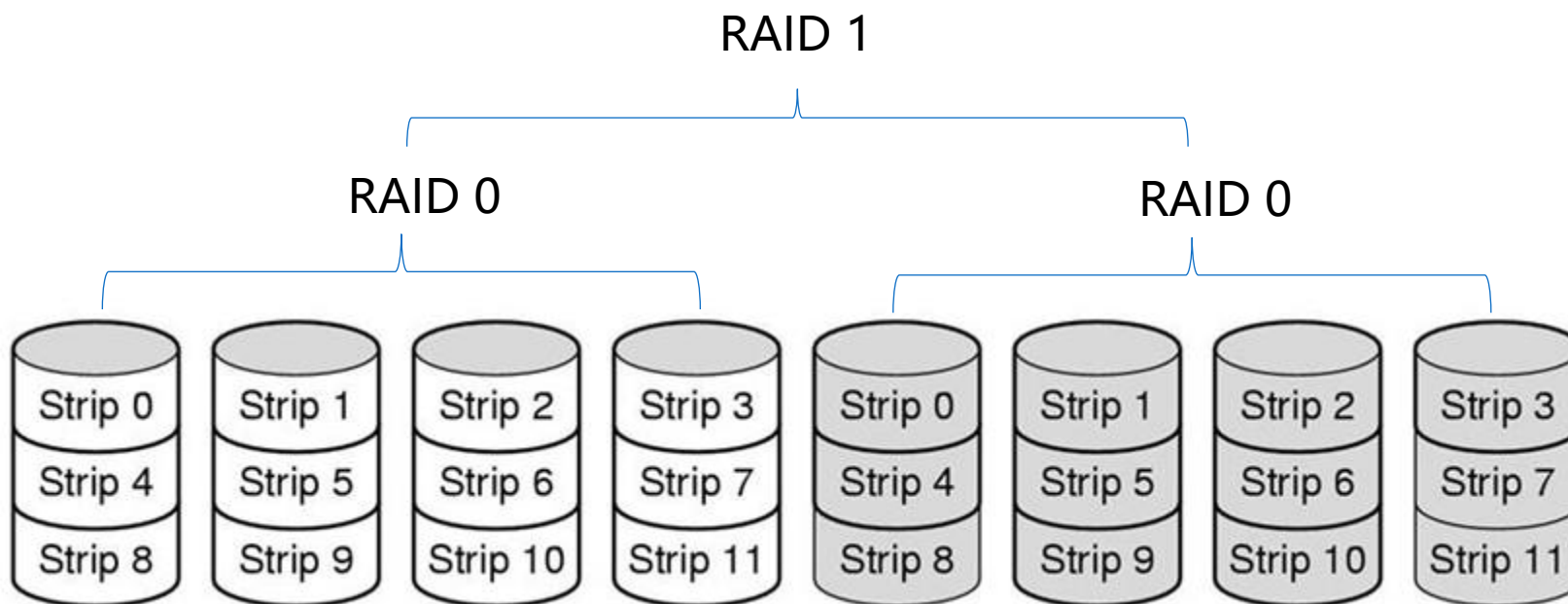
RAID6

- 比RAID5增加了一个额外的校验块
- 组建RAID 6 要求至少4块硬盘， 可以允许坏掉2块硬盘



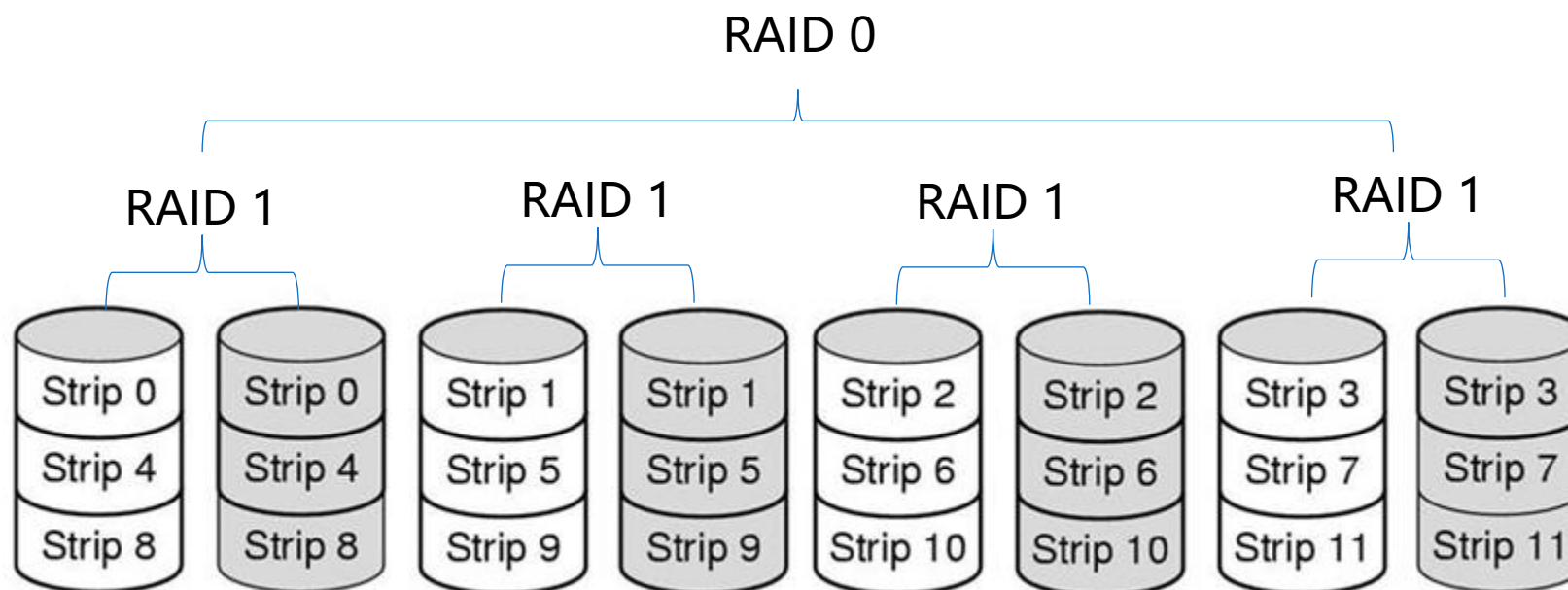
~~RAID 0 + 1 (RAID 01)~~

- RAID 0及RAID 1的结合，采用2组RAID 0的磁盘阵列互为镜像，它们之间又成为一个RAID1的阵列



RAID 1 + 0 (RAID 10)

- RAID 1及RAID 0的结合
- 容错性显著高于RAID 0+1



访问磁盘上的数据时，以下哪一项是主要时间？

- ☐ A 旋转延迟
- ☒ B 寻道时间
- ☐ C 数据传输时间

提交

一块硬盘每个磁道有63个扇区，10个盘片，每个盘片有2个记录盘面和1000个柱面。三元组 (C, H, S) 地址(400, 16, 29)对应于逻辑扇区号

- ☐ A 505035
- ☒ B 505036
- ☐ C 505037
- ☐ D 505038

提交

假设一块硬盘有201个柱面，编号从0到200。在某时刻，磁盘臂位于100柱面处，并且有一个磁盘访问请求队列30、85、90、100、105、110、135和145。如果使用最短寻道时间优先（SSTF）来调度磁盘访问，则在处理_____个请求后，对柱面90的请求进行处理

- ☐ A 1
- ☐ B 2
- ☒ C 3
- ☐ D 4

提交

在存储技术中，RAID代表

- ☐ A Ransomware artificial intelligence defense
- ☒ B Redundant array of independent disks
- ☐ C Random arrangement of interconnected drives
- ☐ D Risk-averse intelligent data

提交

SSD相对于传统硬盘的一个主要优势是什么？

- ☐ A 费用较低
- ☐ B 更大的存储容量
- ☒ C 访问速度更快
- ☐ D 寿命更长

提交

大纲

01

典型的输入/输出设备

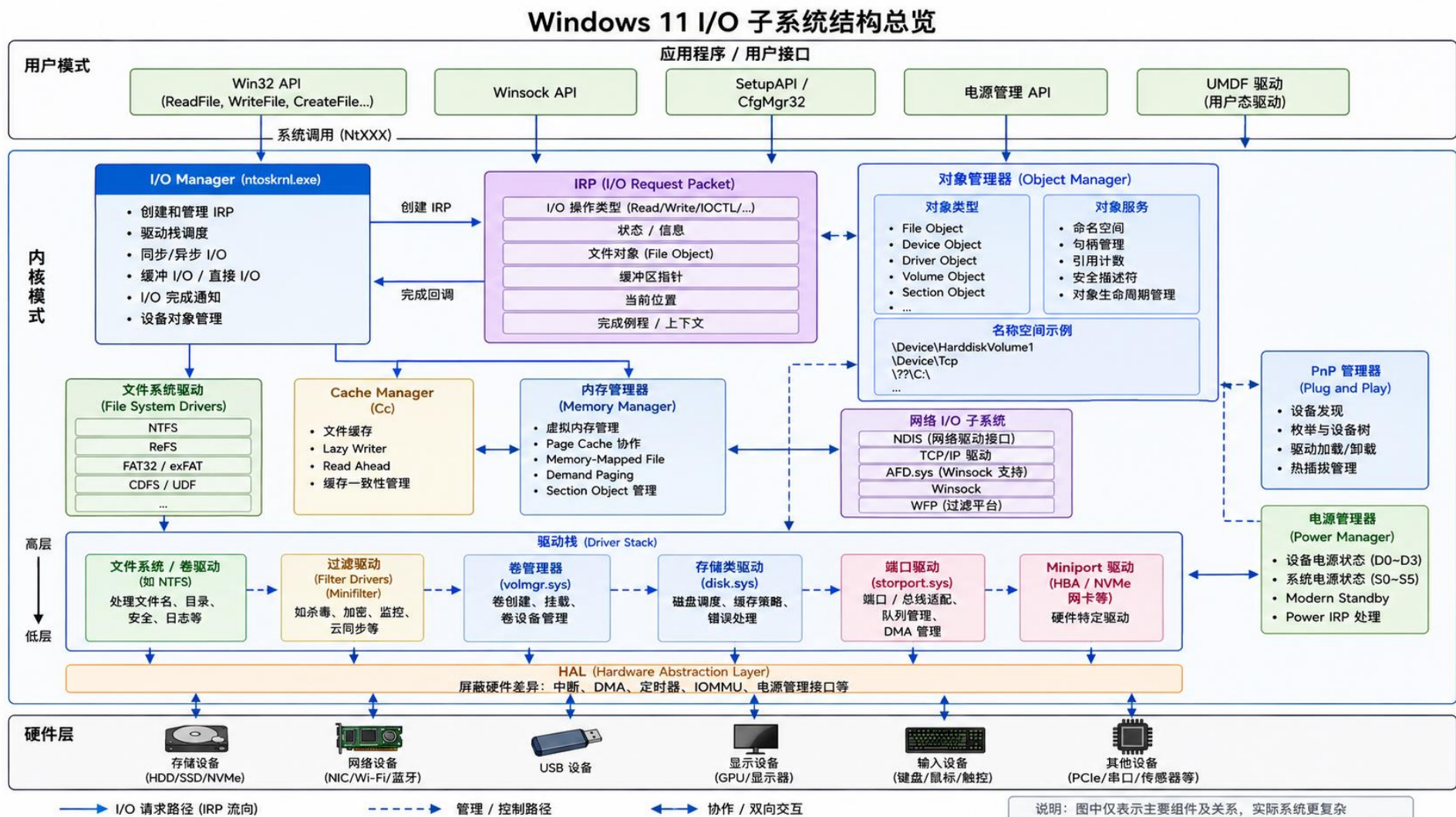
02

Windows的设备管理

03

Linux的设备管理

I/O子系统体系结构



I/O管理器(I/O manager)

- I/O管理器定义有序的工作框架，在该框架里，I/O请求被提交给设备驱动程序
- 大多数I/O请求用“I/O请求包(IRP)”表示，I/O系统是由“包”驱动的，这些包它从一个I/O系统组件移动到另一个I/O系统组件
- I/O管理器创建代表每个I/O操作的IRP，传递IRP给正确的驱动程序，并且当此I/O操作完成后，处理这个数据包
- I/O管理器还为不同的驱动程序提供了公共的代码，驱动程序调用这些代码来执行它们的I/O处理

PnP管理器

- PnP管理器自动识别所有已经安装的硬件设备
- PnP管理器通过一个名为资源仲裁(resource arbitrating)的进程收集硬件资源需求(中断, I/O地址等)来实现硬件资源的优化分配, 满足系统中的每一个硬件设备的资源需求
- PnP管理器通过硬件标识选择应该加载的设备驱动程序
- PnP管理器也为检测硬件配置变化提供了应用程序和驱动程序的接口, 因此在Windows中, 在硬件配置发生变化时, 相应的应用程序和驱动程序也会得到通知

电源管理器

- 电源管理需要底层硬件符合ACPI标准
- ACPI为系统和设备定义了不同的能耗状态
 - 系统:从S0(正常工作)到S5(完全关闭)
 - 设备:从D0(正常工作)到D4(完全关闭)
- 电源消耗: 计算机系统消耗的能源
- 软件运行恢复: 计算机系统回复到正常工作状态时软件能否恢复运行
- 硬件延迟: 计算机系统回复到正常工作状态的时间延迟

Windows电源管理策略

- 电源管理器是系统电源策略的所有者，因此整个系统的能耗状态转换由电源管理器决定，并调用相应设备的驱动程序完成，电源管理器根据以下因素决定当前的能耗状态
 - 系统活动状况
 - 系统电源状况
 - 应用程序的关机、休眠请求
 - 用户的操作，例如用户按电源按钮
 - 控制面板的电源设置
- 设备驱动程序可以独立地控制设备的能耗状态

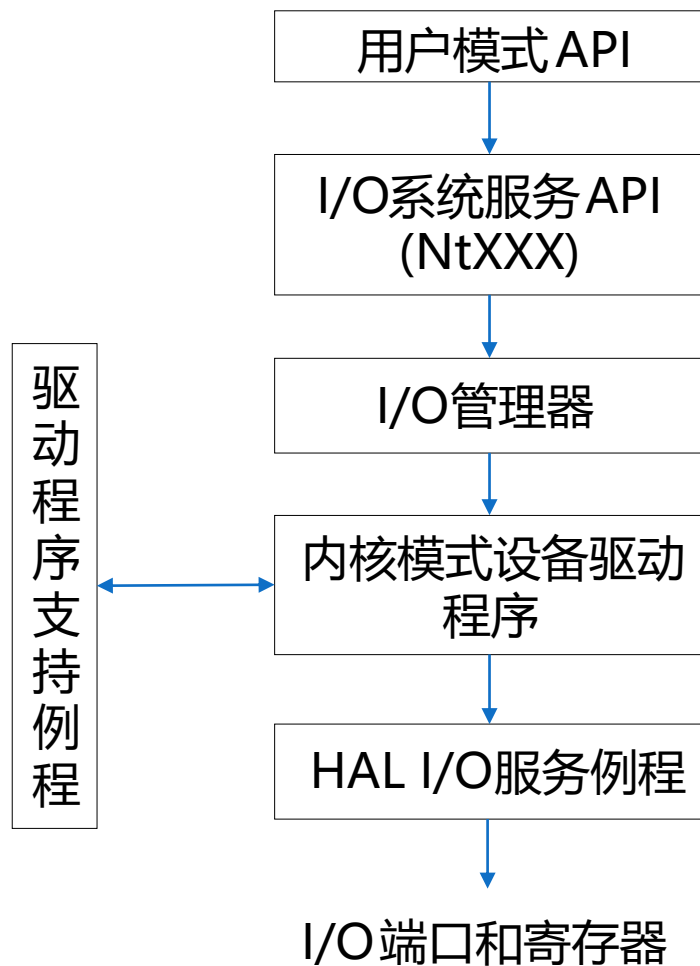
设备驱动程序

- 设备驱动程序为某种类型的设备提供一个I/O接口。设备驱动程序从I/O管理器接受处理命令，当处理完毕后通知I/O管理器
- 设备驱动程序之间的协同工作也通过I/O管理器进行

Windows I/O的特点

- 在Windows中，所有的I/O操作都通过虚拟文件执行，隐藏了I/O操作目标的实现细节，为应用程序提供了一个统一的到设备的接口界面
- 用户态应用程序调用文档化的函数，这些函数再依次地调用内部I/O子系统函数来从文件中读取、对文件写入和执行其他的操作。I/O管理器动态地把这些虚拟文件请求指向适当的设备驱动程序

典型的I/O请求过程

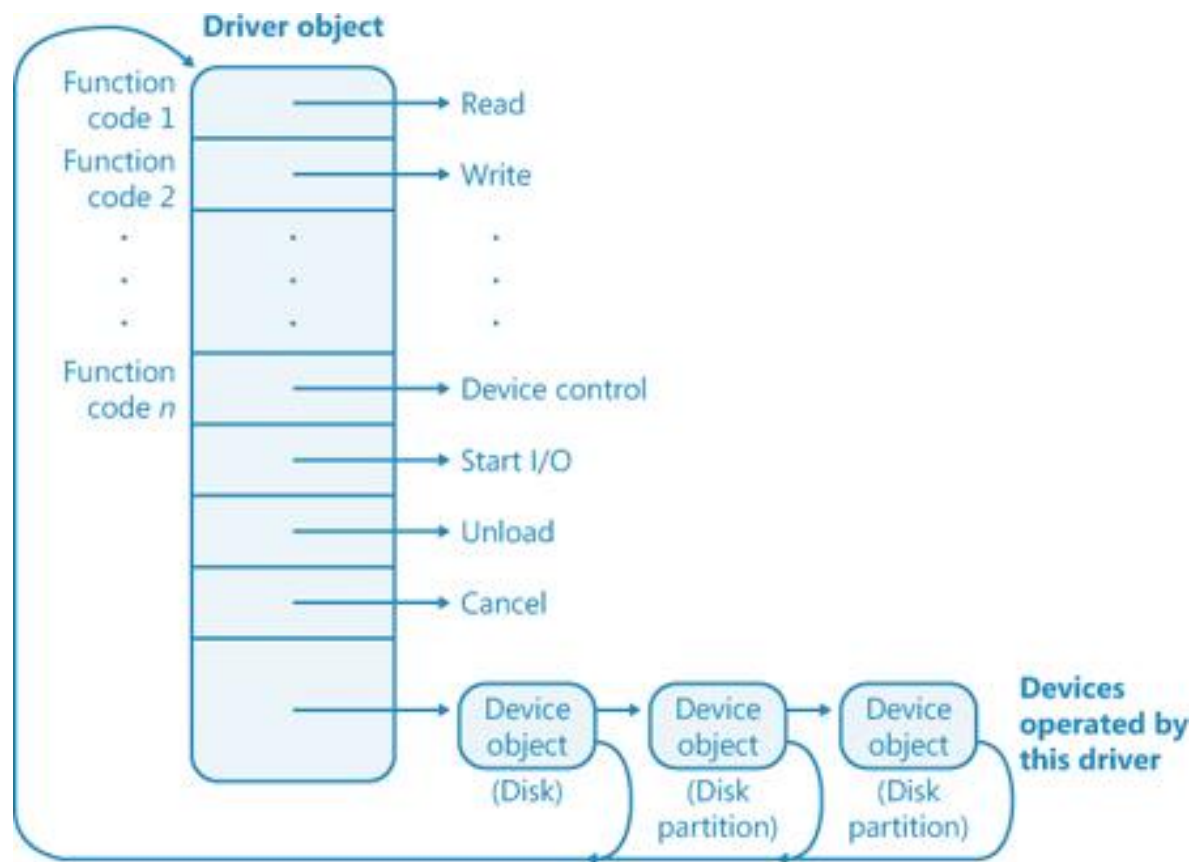


重要的I/O系统对象

- **驱动程序对象**在系统中代表一个独立的驱动程序，并且为I/O记录每个驱动程序的调度例程的地址(入口点)
- **设备对象**在系统中代表一个物理的、逻辑的或虚拟的设备并描述了它的特征

重要的I/O系统对象

■ 驱动程序对象和设备对象



重要的I/O系统对象

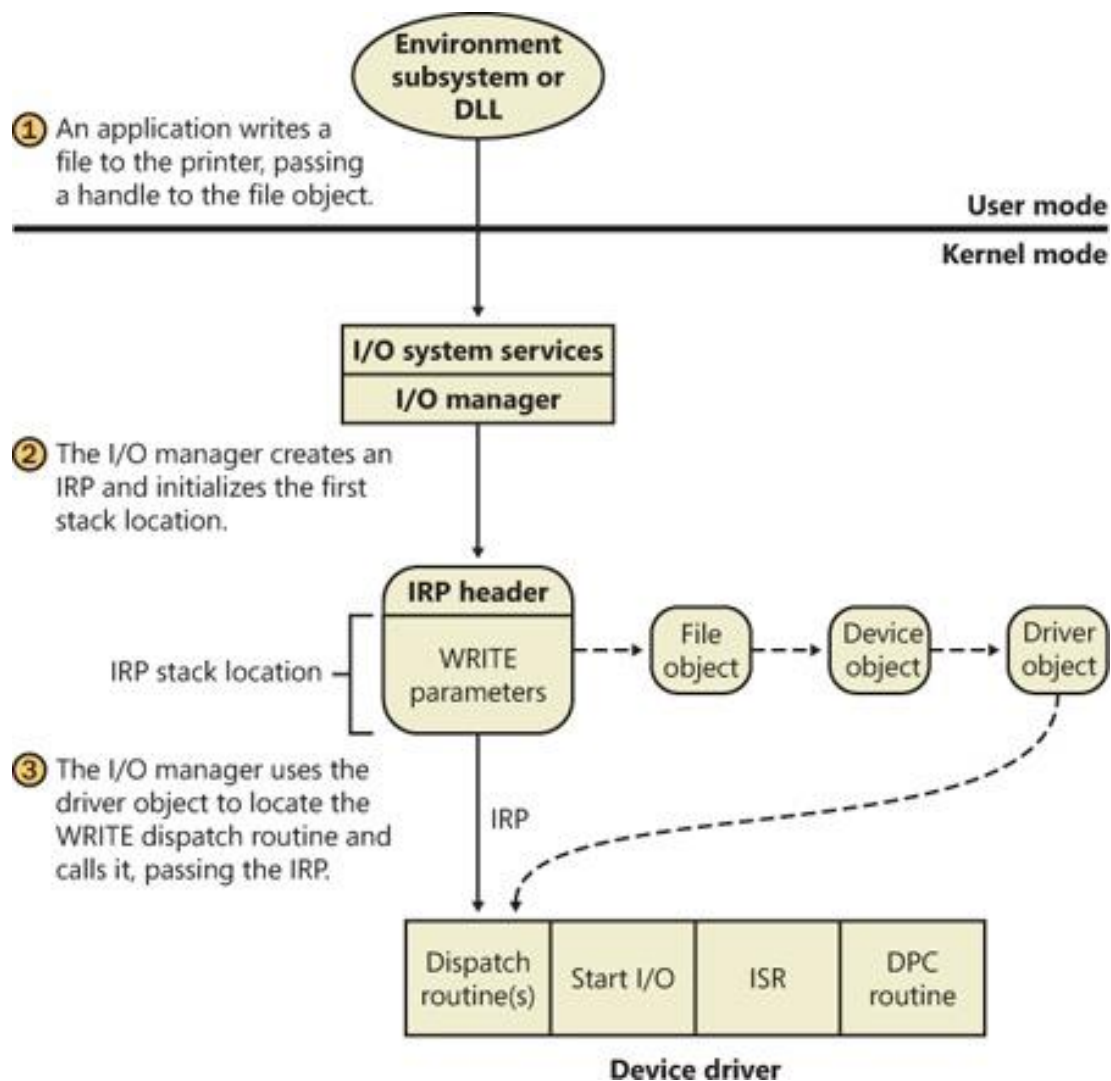
- I/O请求包存储处理I/O请求所需信息
- 线程调用I/O服务时，I/O管理器就构造一个I/O请求包(IRP, I/O Request Package)来表示在整个系统I/O进展中要进行的操作
- I/O管理器在IRP中保存一个指向调用者文件对象的指针

重要的I/O系统对象

- IRP由两部分组成：
- 固定部分(称作标题)：请求的类型和大小、是同步请求还是异步请求、用于缓冲I/O的指向缓冲区的指针和随着请求的进展而变化的状态信息
- 一个或多个堆栈单元：一个功能码、功能特定的参数和一个指向调用者文件对象的指针

重要的I/O系统对象

I/O系统对象的关系



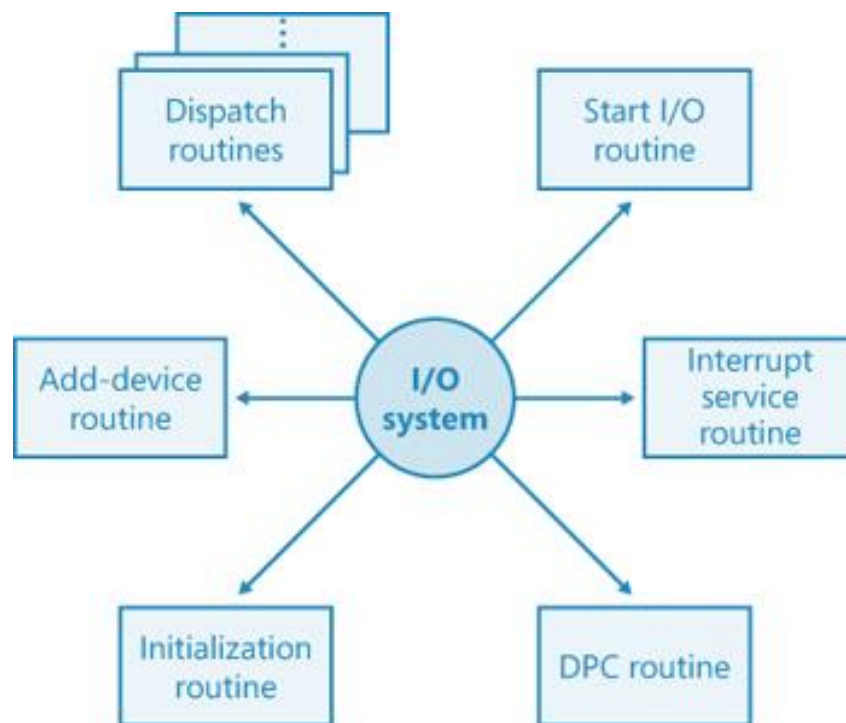
Windows设备驱动程序

- 支持多种类型的设备驱动程序和编程环境
- 硬件支持驱动程序可以分类如下
 - 类驱动程序(class drivers)为某一类设备执行I/O处理，例如磁盘、磁带或光盘
 - 端口驱动程序(port drivers)实现了对特定于某一种类型的I/O端口的I/O请求的处理，例如SCSI
 - 小端口驱动程序(miniport drivers)把对端口类型的一般的I/O请求映射到适配器类型。例如，一个特定的SCSI适配器

Windows设备驱动程序

■ 设备驱动程序包括一组被调用处理I/O请求不同阶段的例程

- 调度例程
- 启动I/O例程
- 中断服务例程
- DPC例程
- 初始化例程
- 添加设备例程



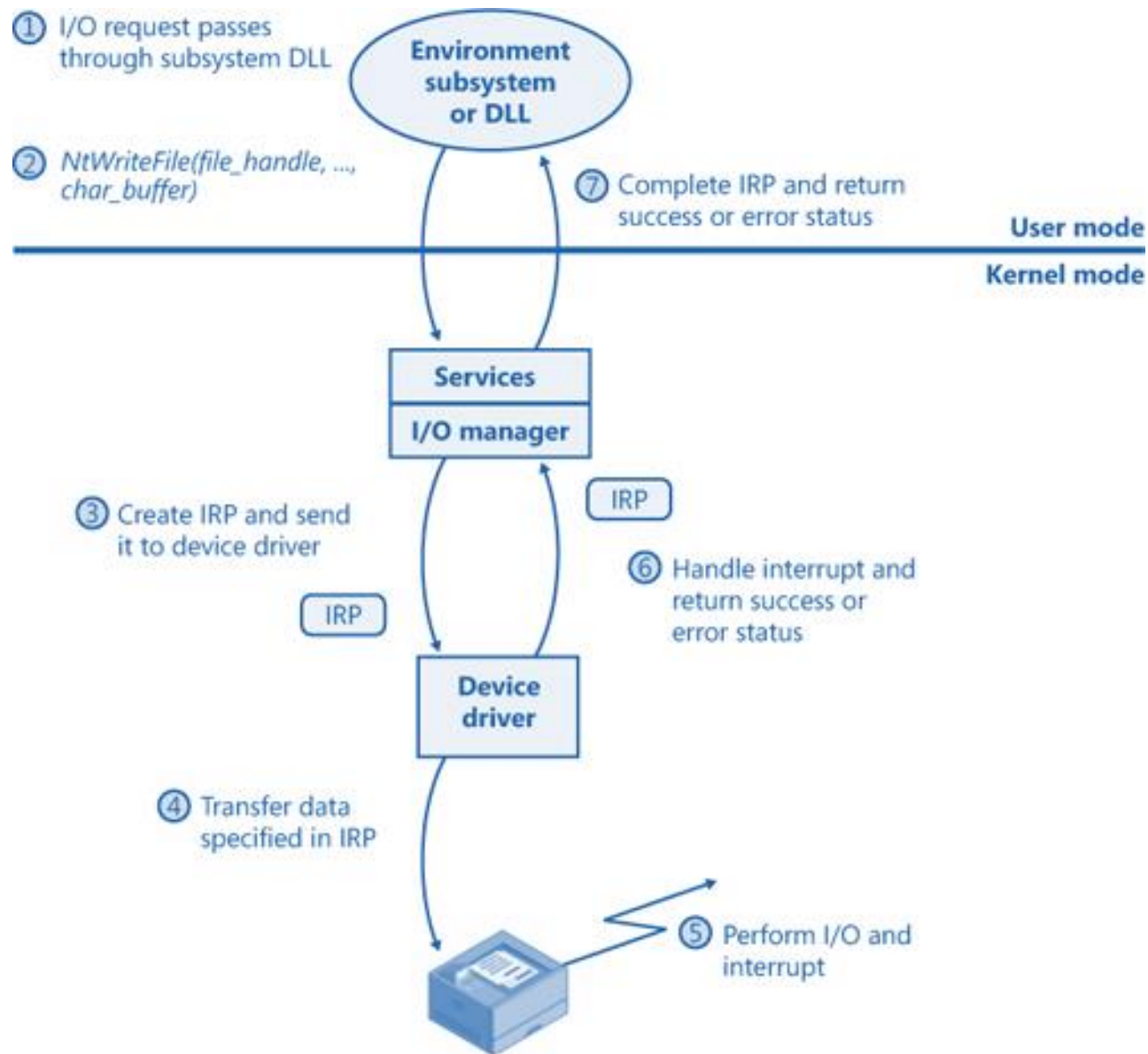
Windows的I/O处理

■对单层驱动程序的I/O请求处理

1. I/O请求经过子系统DLL
2. 子系统DLL调用I/O管理器的服务
3. I/O管理器以IRP的形式给驱动程序(这里指设备驱动程序)发送请求
4. 驱动程序启动I/O操作
5. 设备执行I/O操作，完成操作时发出中断
6. 设备驱动程序服务于中断
7. I/O管理器完成I/O请求

Windows的I/O处理

■对单层驱动程序的I/O请求处理

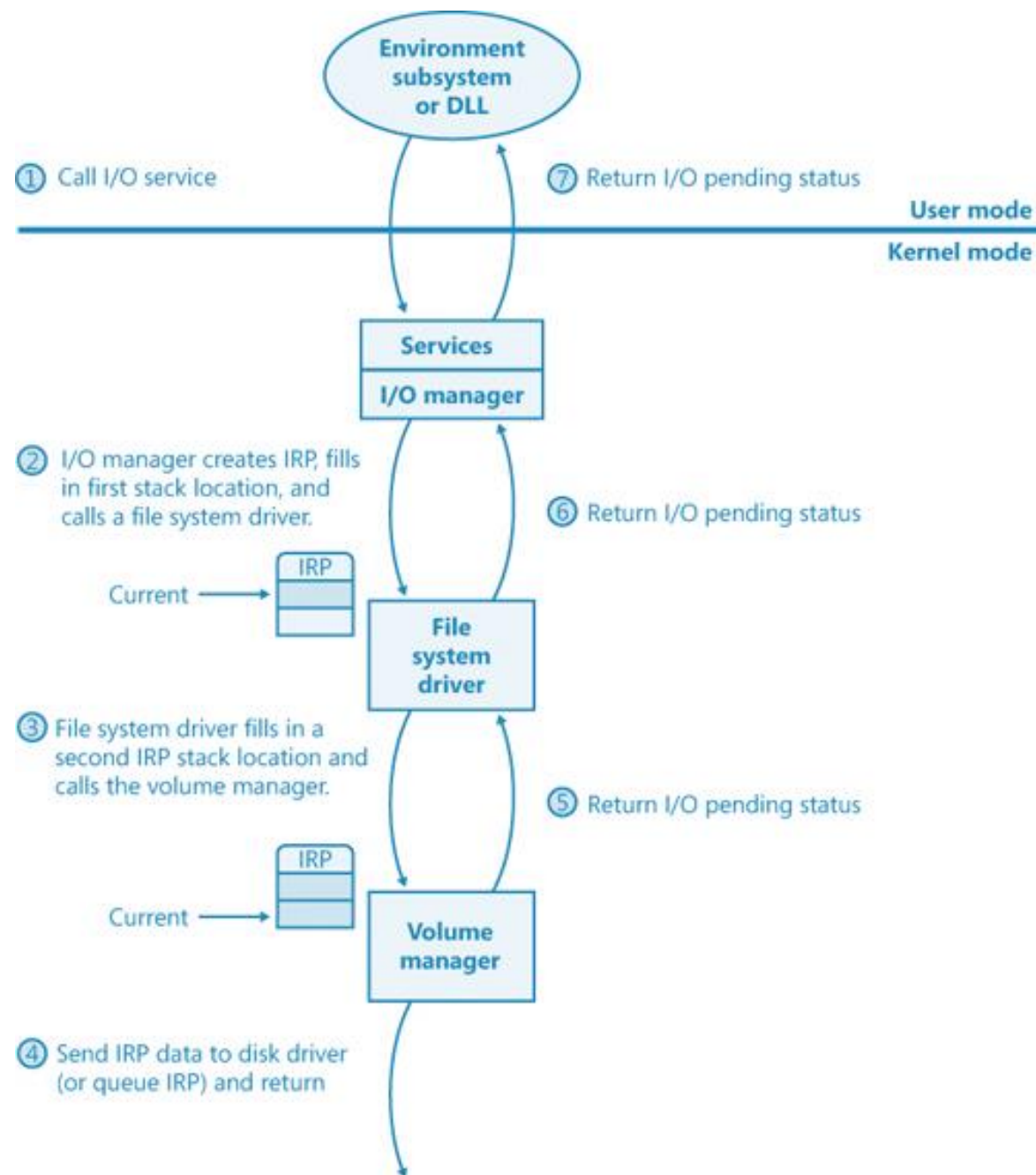


Windows的I/O处理

- 对多层驱动程序的I/O请求处理
- 在单层I/O处理的基础上变化而来
- I/O管理器调用顶层设备驱动程序
- 然后上层设备驱动程序调用低一级的驱动程序，形成I/O请求的转换、传递和嵌套。最终形成对设备的操作
- 分层驱动程序多用于几个设备的协作或者像文件、网络这样的复杂功能实体中

Windows的I/O处理

■对多层驱动程序的I/O请求处理



大纲

01

典型的输入/输出设备

02

Windows的设备管理

03

Linux的设备管理

Linux的设备管理

- Linux系统设备管理的基本特点是抽象了设备处理，所有对硬件设备的操作和通常的文件一样，利用标准的系统调用可在设备上打开、关闭、读取或写入操作
- 系统中的每个设备由设备特殊文件(special file)来代表。例如，`/dev/hda` 代表系统中的第一个IDE硬盘，而 `/dev/sda1` 则代表系统中SCSI硬盘上的第一个分区

Linux的设备管理

- Linux支持三种设备：字符设备、块设备及网络设备
- **字符设备**指那些无需缓冲直接读写的设备，如系统的串口设备/dev/cua0和/dev/cua1
- **块设备**则仅能以块为单位读写，典型的块大小为512或1024字节。块设备的存取是通过buffer cache来进行并且可以进行随机访问，即不管块位于设备中何处都可以对其进行读写。块设备可以通过其设备相关文件进行访问，但更为平常的访问方法是通过文件系统。只有块设备才能支持可安装文件系统
- **网络设备**则可以通过BSD套接字进行访问

Linux的设备管理

- 块设备和字符设备的设备特殊文件可以通过mknod命令来创建，并使用主、从设备号来描述此设备
- 网络设备也用设备相关特殊文件来表示，但Linux寻找和初始化网络设备时才建立这种文件

Linux的设备管理

- 在多任务操作系统中，需要内核建立应用程序和设备之间的抽象接口，而不是由应用程序直接操作硬件。为此，操作系统一般提供设备驱动程序来专门完成对特定硬件的控制
- 设备驱动程序实际是处理或操作硬件控制器的软件，从本质上讲，它们是内核中具有高特权级的、驻留内存的、可共享的底层硬件处理例程

Linux的设备管理

- 每个特殊文件都与驱动程序相关联来处理相应的设备
- 由同一个设备驱动控制的所有设备具有相同的主设备号。从设备号则被用来区分具有相同主设备号且由同一个设备驱动程序控制的不同设备。例如主IDE硬盘的每个分区的从设备号都不相同
- 设备特殊文件的VFS索引节点中包含设备号信息。如果通过系统调用访问设备，则内核可通过该VFS索引节点中的设备号信息调用适当的设备驱动程序

设备驱动程序

- Linux内核中虽存在许多不同的设备驱动程序，但它们具有一些共性：
 1. **内核代码**：设备驱动程序是内核的一部分，如同内核中其它代码一样，出错将导致系统的严重损害。编写得很差的设备驱动程序甚至能使系统崩溃并导致文件系统的破坏和数据丢失
 2. **内核接口**：设备驱动程序必须为Linux内核或者其从属子系统提供一个标准接口。例如终端驱动为Linux内核提供了一个文件I/O接口而SCSI设备驱动为SCSI子系统提供了一个SCSI设备接口，同时此子系统为内核提供了文件I/O和buffer cache接口
 3. **内核机制与服务**：设备驱动程序可以使用标准的内核服务如内存分配、中断发送和等待队列等等

设备驱动程序

- 4. **动态可加载**: 多数Linux设备驱动程序可以在内核模块发出加载请求时加载, 同时在不再使用时卸载。这样内核能有效地利用系统资源
- 5. **可配置**: Linux设备驱动程序可以连接到内核中。当内核被编译时, 可配置决定哪些内核被连入内核
- 6. **动态性**: 当系统启动及设备驱动程序初始化时将查找它所控制的硬件设备。如果某个设备的驱动程序为一个空过程并不会有什么问题。此时此设备驱动程序仅仅是一个冗余的程序, 它除了会占用少量系统内存外不会对系统造成什么危害

设备驱动程序和内核的接口

- 每种类型的驱动程序，不管它们是字符设备、块设备还是网络设备，均为内核提供相同的调用接口。于是，内核可以以相同的方式处理不同的设备。例如，内核可通过相同的函数调用让SCSI和IDE硬盘完成相同的工作
- 为了实现这些特点，Linux为每种不同类型的设备驱动程序维护相应的数据结构，以便定义统一的接口并实现驱动程序的可装载性、动态性等

字符设备

- 字符设备是Linux设备中最简单的一种。应用程序可以和存取文件相同的系统调用来打开、读写及关闭它
- Linux内核中所有已分配的字符设备都记录在一个名为 `chrdevs` 散列表里
- `chrdevs` 散列表的大小是 255，散列算法是把每组字符设备编号范围的主设备号以 255 取模插入相应的散列桶中

字符设备

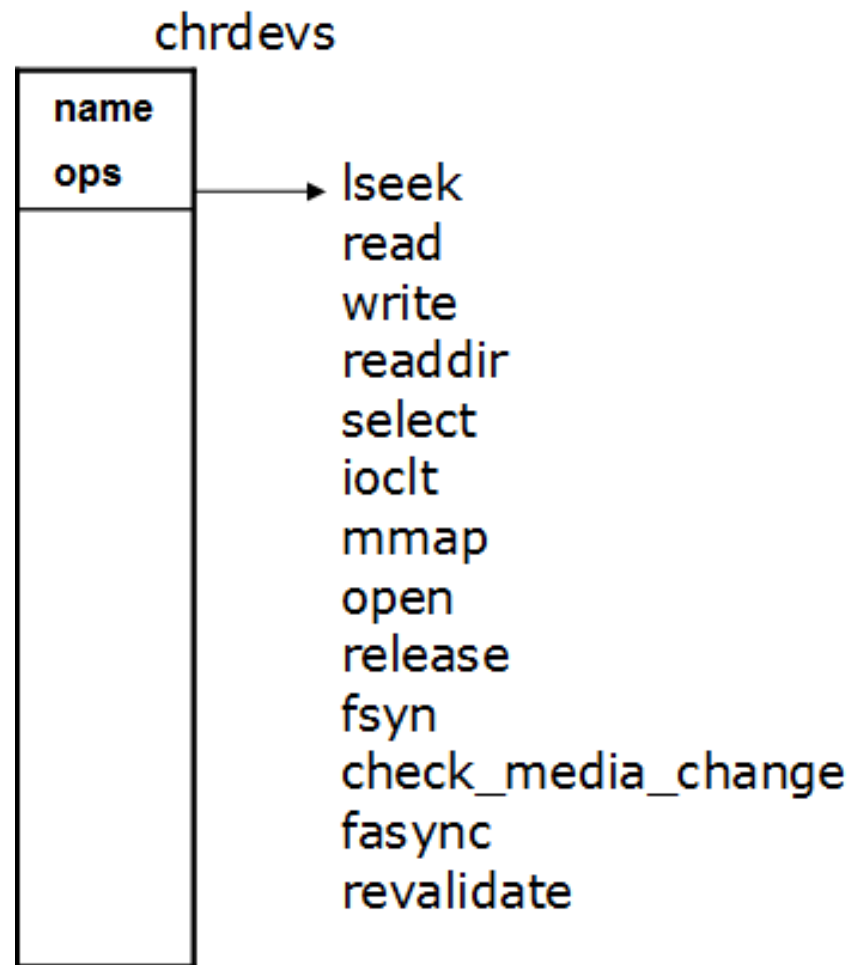
- 针对散列冲突问题，next字段是冲突链表中的下一个元素的指针
- 内核内部使用cdev结构来表示字符设备

```
static struct char_device_struct {  
    struct char_device_struct *next;  
    unsigned int major;  
    unsigned int baseminor;  
    int minorct;  
    char name[64];  
    struct cdev *cdev; /* will die */  
} *chrdevs[CHRDEV_MAJOR_HASH_SIZE];
```

```
struct cdev {  
    struct kobject kobj;  
    struct module *owner;  
    const struct file_operations *ops;  
    struct list_head list;  
    dev_t dev;  
    unsigned int count;  
};
```

字符设备

- name是设备驱动程序注册的设备名称
- ops包含设备的标准操作例程集，其中的操作例程由设备驱动程序定义，并分别完成指定的设备操作



字符设备

- 当打开代表字符设备的字符特殊文件时(如/dev/cua0)，操作系统内核必须作好准备以便调用相应字符设备驱动的文件操作例程
- 与普通的目录和文件一样，每个字符特殊文件用一个虚拟文件系统(Virtual File System, VFS)的节点(inode)表示。每个字符特殊文件使用的VFS inode和所有设备特殊文件一样，包含着设备的主从设备号。这个VFS inode由底层的文件系统来建立，其信息来源于设备特殊文件名称所在文件系统

字符设备

- 当代表某个字符设备的设备特殊文件被打开时，内核负责进行一些设置工作，以便能够调用正确的设备操作例程
- 和其他普通的文件或目录一样，设备文件也由 VFS 索引节点代表。VFS 索引节点中包含设备的主设备号和次设备号，每个 VFS 节点和一组文件操作函数的指针关联

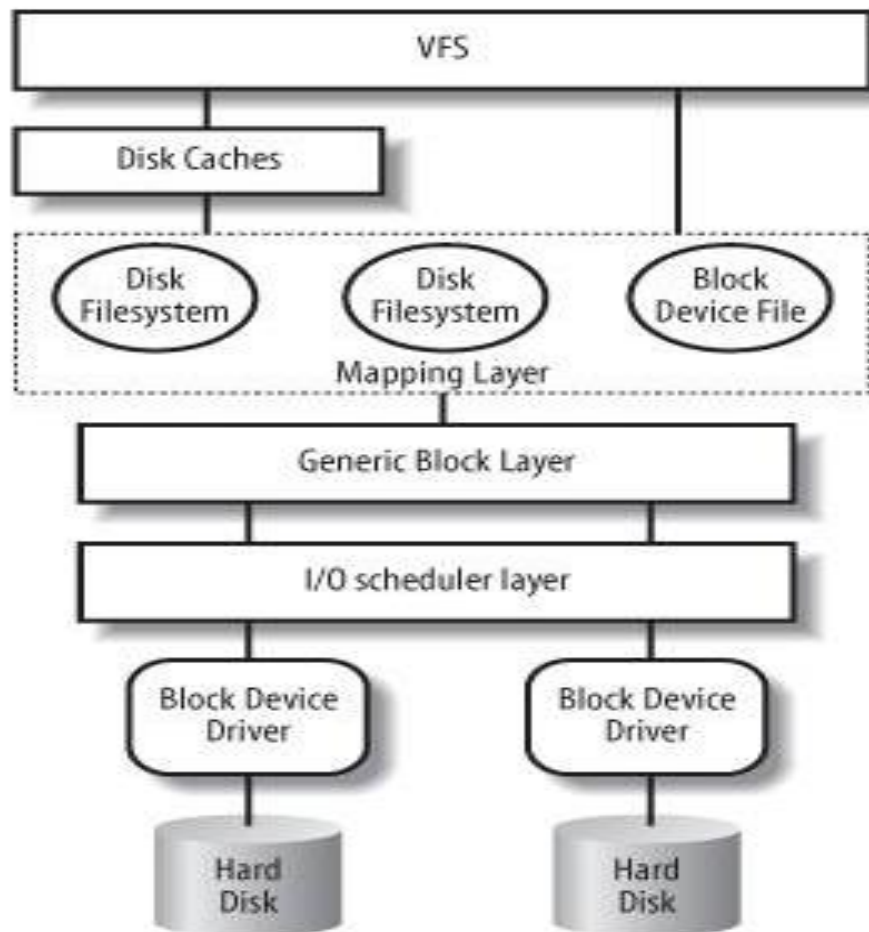
字符设备

- 系统在建立一个代表字符设备文件的VFS索引节点时，该VFS节点的文件操作函数指针被设置为默认的字符设备操作函数指针。这时实际只有一个文件操作函数被定义，即文件的打开操作
- 应用程序打开某个字符特殊文件时，该打开操作将使用VFS节点中的主设备号在chrdevs数组中检索相应的设备操作函数地址，同时在进程的文件数据结构中建立一个file结构，该结构的文件操作函数指针指向设备驱动程序定义的设备操作函数指针
- 此后，应用程序在该设备特殊文件上进行的读取、写入等操作就映射到了特定的字符设备操作

块设备

- Linux中，块设备主要为存储设备如硬盘、Flash而设计
- 块设备可随机地以固定大小的块传送数据
- 用户并不能象字符设备那样直接在用户程序中访问，而是通过文件系统在访问文件时间接访问

块设备



块设备

■ 通用块层(Generic Block Layer)

- 隐藏硬件细节，提供block设备的抽象视图
- 提供通用的数据结构描述disks和disk partitions

■ I/O调度器(I/O Scheduler)

- 根据内核制定的策略对未决的(pending) I/O数据传送请求进行排序和调度
- 提高I/O调度器的效率也是影响整个系统对块设备上数据管理效率的一个方面

■ 块设备驱动程序(Block Device Driver)

- 完成和硬件的具体交互

块设备

- 每个块设备都是由一个block_device结构的描述符来表示
- 所有的块设备描述符组成一个全局链表，链表头由变量all_bdevs表示；链表链接所用的指针位于块设备描述符的bd_list字段中

块设备

- 如果块设备描述符对应一个磁盘分区，那么bd_contains字段指向与整个磁盘相关的块设备描述符，而bd_part字段指向hd_struct分区描述符
- 否则，若块设备描述符对应整个磁盘，那么bd_contains字段指向块设备描述符本身

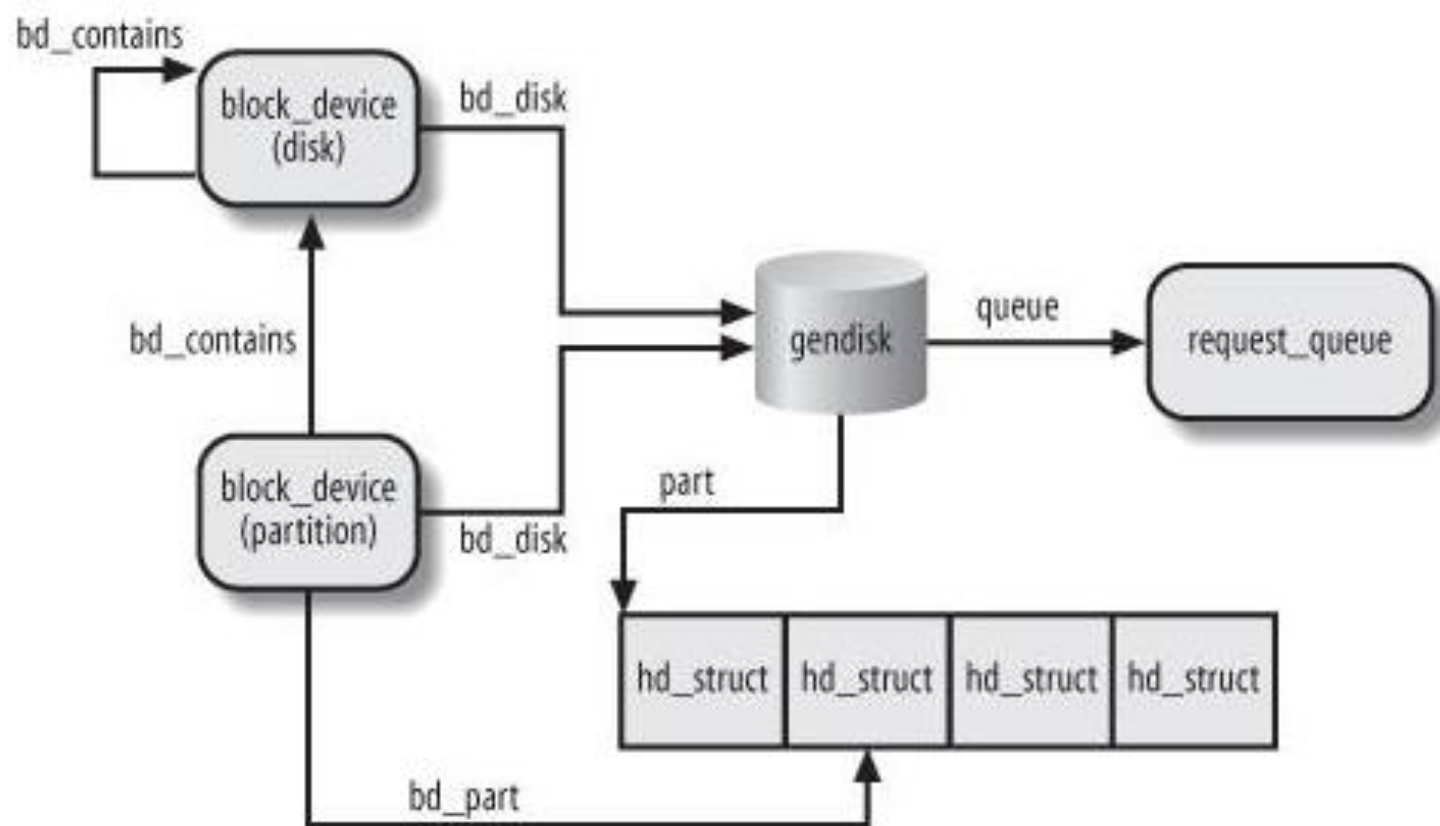
```
struct block_device {
    dev_t bd_dev;
    struct inode * bd_inode;
    int bd_openers;
    struct mutex bd_mutex;
    struct mutex bd_mount_mutex;
    struct list_head bd_inodes;
    void * bd_holder;
    int bd_holders;
#ifdef CONFIG_SYSFS
    struct list_head bd_holder_list;
#endif
    struct block_device * bd_contains;
    unsigned bd_block_size;
    struct hd_struct * bd_part;
    unsigned bd_part_count;
    int bd_invalidated;
    struct gendisk * bd_disk;
    struct list_head bd_list;
    struct backing_dev_info
    *bd_inode_backing_dev_info;
    unsigned long bd_private;
};
```

块设备

- 每一个块设备物理实体由一个gendisk结构体来表示，每个gendisk可以支持若干个分区
- 每个gendisk中包含本物理实体的全部信息及操作函数接口

```
struct gendisk {  
    int major;                                /* major number of driver */  
    int first_minor;  
    int minors;                               /* maximum number of minors, =1 for disks that can't be partitioned. */  
    char disk_name[32];                       /* name of major driver */  
    struct hd_struct **part;                  /* [indexed by minor] */  
    int part_uevent_suppress;  
    struct block_device_operations *fops;  
    struct request_queue *queue;  
    void *private_data;  
    sector_t capacity;  
    int flags;  
    struct device *driverfs_dev;  
    struct kobject kobj;  
    struct kobject *holder_dir;  
    struct kobject *slave_dir;  
    struct timer_rand_state *random;  
    int policy;  
    atomic_t sync_io;                         /* RAID */  
    unsigned long stamp;  
    int in_flight;  
    #ifdef CONFIG_SMP  
    struct disk_stats *dkstats;  
    #else  
    struct disk_stats dkstats;  
    #endif  
}
```

块设备



块设备

- 多个 request 结构形成了一个链表，而每个 request 结构代表由缓冲区高速缓存请求设备读取或写入的请求信息
- 每当缓冲区高速缓存要从注册设备中读取数据或向设备写入数据时，缓冲区高速缓存就会在gendisk中添加一个 request结构

网络设备

- 网络设备是一个发送与接收数据包的实体。它一般是一个像以太网卡的物理设备。有些网络设备如loopback设备仅仅是一个用来向自身发送数据的软件
- 每个网络设备都用一个device结构来表示。网络设备驱动在内核启动初始化网络时将这些受控设备登记到Linux中

网络设备

- device数据结构中包含有有关设备的信息以及用来支持各种网络协议的函数地址指针。这些函数主要用来使用网络设备传输数据。设备使用标准网络支持机制来将接收到的数据传递到适当的协议层
- 所有传输与接收到的网络数据用一个sk_buff结构表示，这些灵活的数据结构使得网络协议头可以更容易的添加与删除

网络设备的数据结构device

- **Name**: 与使用mknod命令创建的块设备特殊文件与字符设备特殊文件不同，网络设备特殊文件仅在于系统网络设备发现与初始化时建立。它们使用标准的命名方法，每个名字代表一种类型的设备。多个相同类型设备将从0开始记数。这样以太网设备被命名为/dev/eth0, /dev/eth1等等

/dev/ethN	以太网设备
/dev/slN	SLIP设备
/dev/pppN	PPP 设备
/dev/lo	Loopback 设备

网络设备的数据结构device

- **Bus Information**: 这些信息被设备驱动用来控制设备。irq号表示设备使用的中断号。base address指任何设备在I/O内存中的控制与状态寄存器地址。DMA通道指此网络设备使用的DMA通道号。所有这些信息在设备初始化时设置
- **Interface Flags**: 它们描述了网络设备的属性与功能
- **Protocol Information**: 每个设备描述它可以被网络协议层如何使用

网络设备的数据结构device

- **Bus Information**: 这些信息被设备驱动用来控制设备。irq号表示设备使用的中断号。base address指任何设备在I/O内存中的控制与状态寄存器地址。DMA通道指此网络设备使用的DMA通道号。所有这些信息在设备初始化时设置
- **Interface Flags**: 它们描述了网络设备的属性与功能
- **Protocol Information**: 每个设备描述它可以被网络协议层如何使用

操作系统，2026

谢谢

OPERATING SYSTEM